

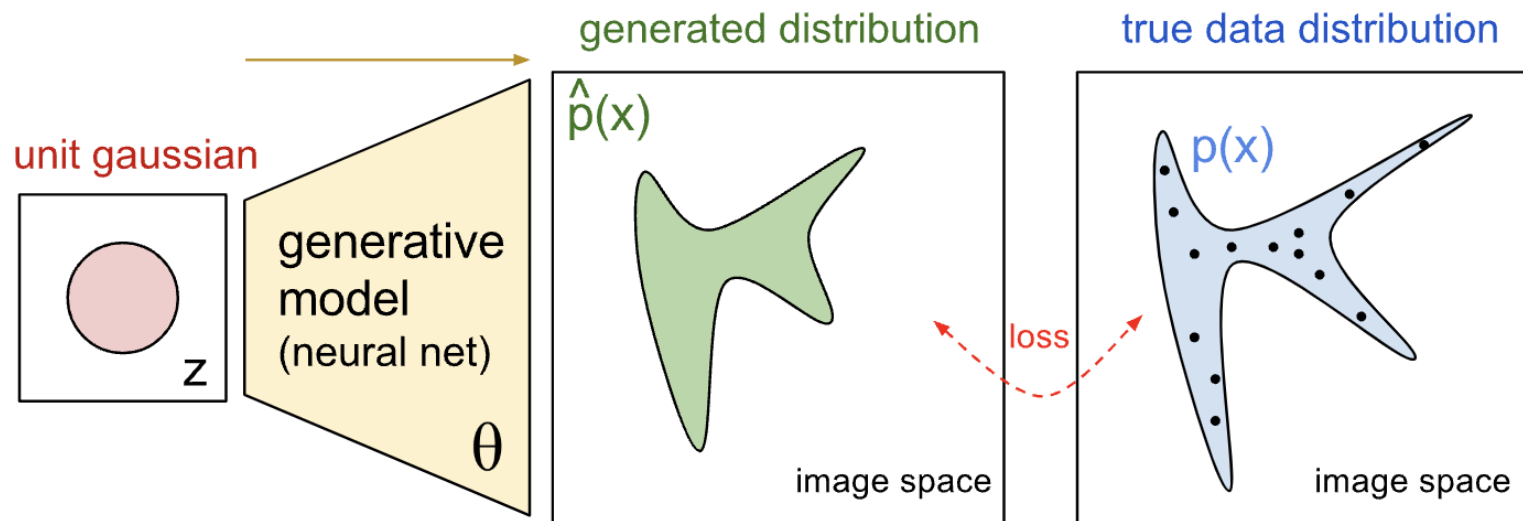
AMA 564 Deep Learning

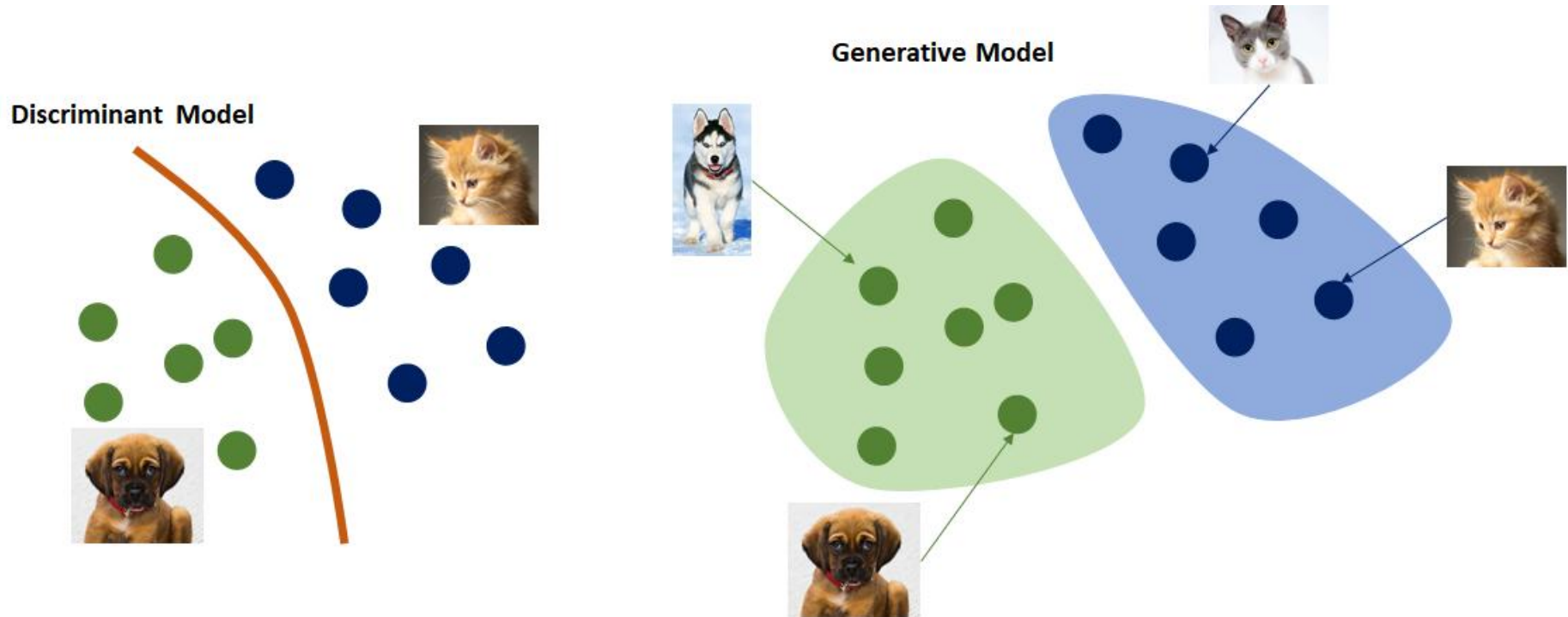
2026 Spring

Lecture 7

Generative Models

1. A generative model is a type of machine learning model that learns the probability distribution of a dataset and generates new samples that are similar to the original data.
2. It learns how to generate samples by modelling the underlying structure and patterns of the training data, and can be used for tasks such as image generation, text generation, and speech synthesis.





Source:<https://medium.com/@jordi299/about-generative-and-discriminative-models-d8958b67ad32>

Discriminative models

learning the decision boundary between different classes.

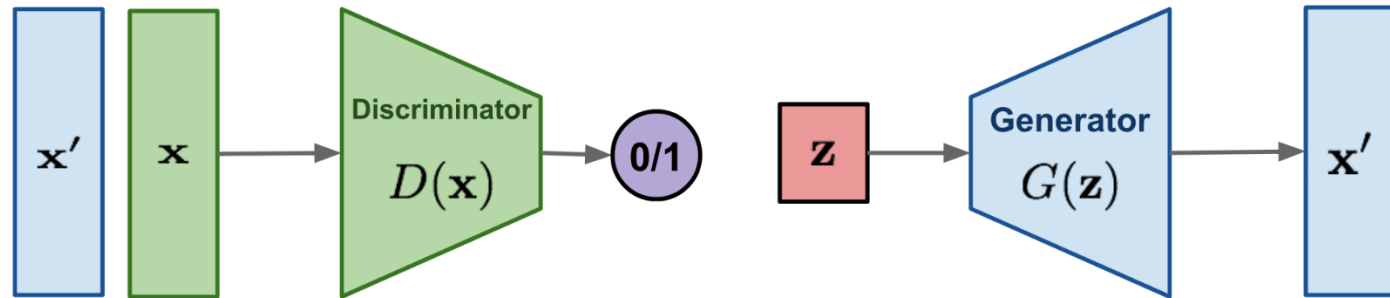
Generative models

model the probability distribution of the entire dataset.

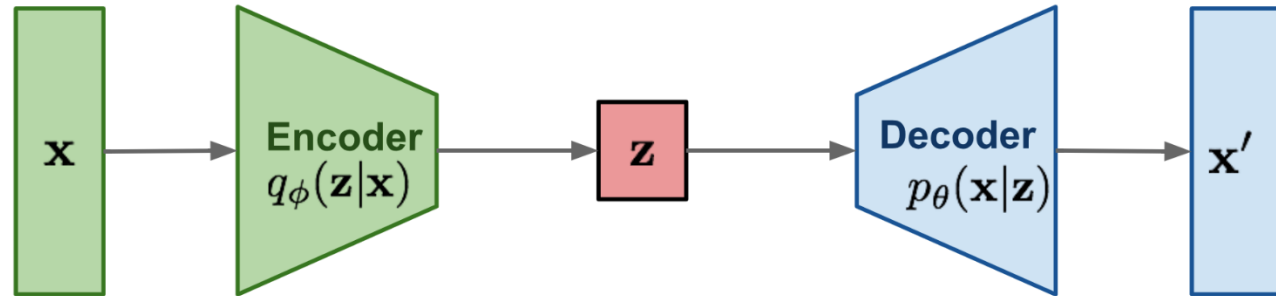
Generative models



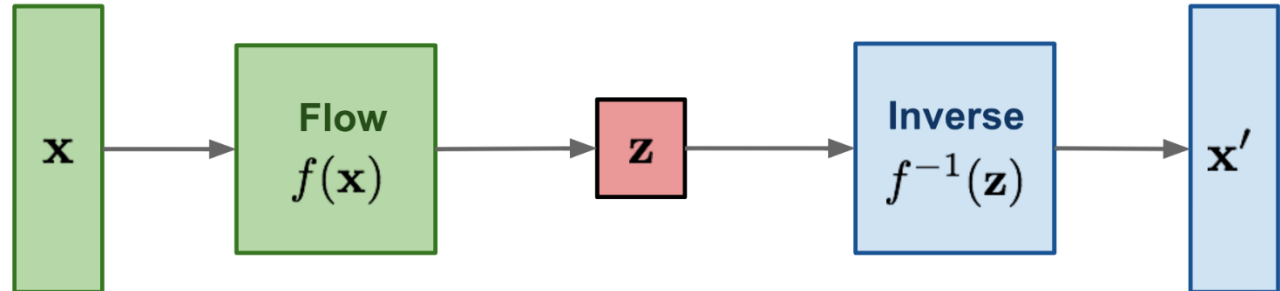
GAN: minimax the classification error loss.



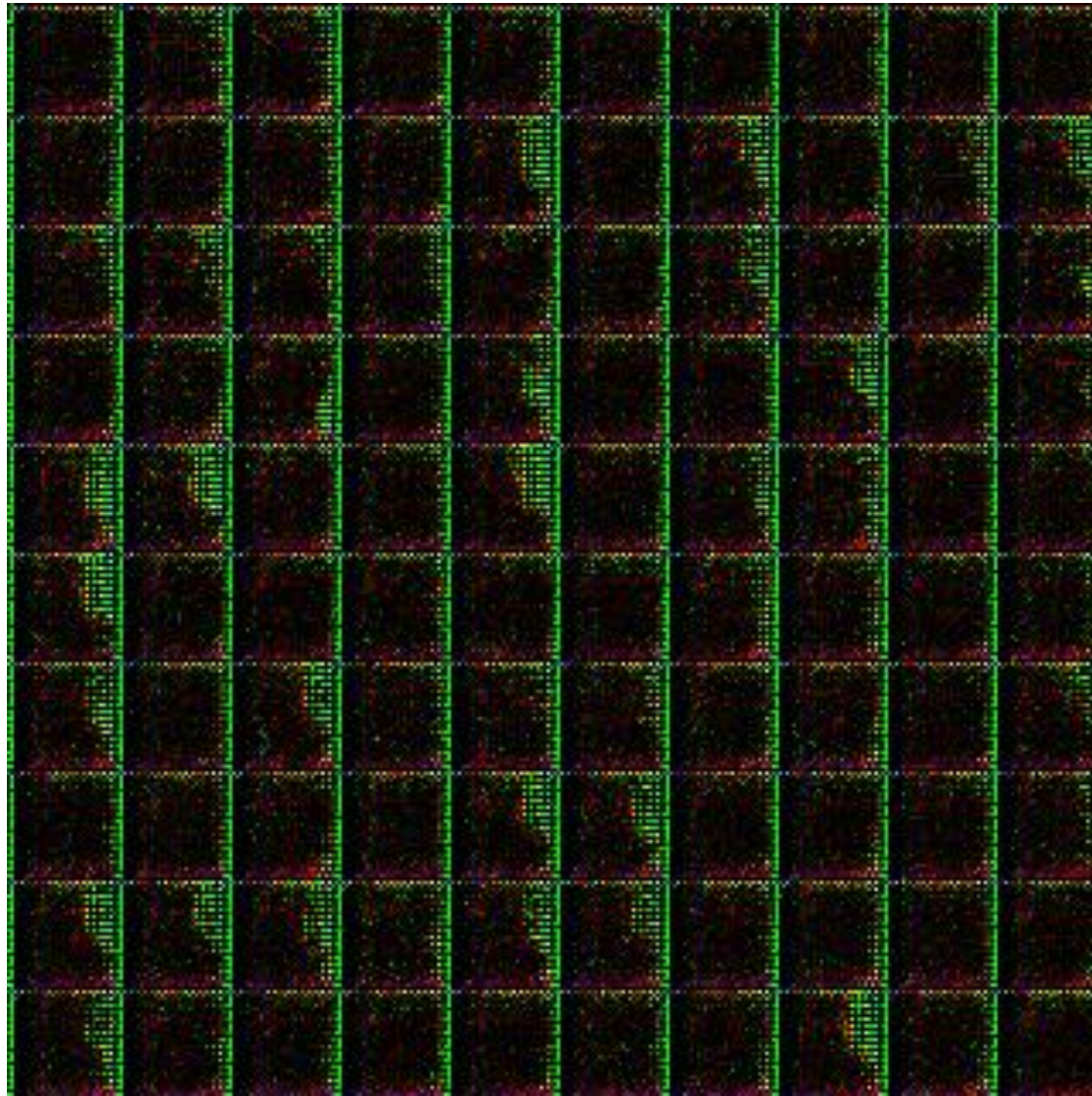
VAE: maximize ELBO.



Flow-based generative models: minimize the negative log-likelihood



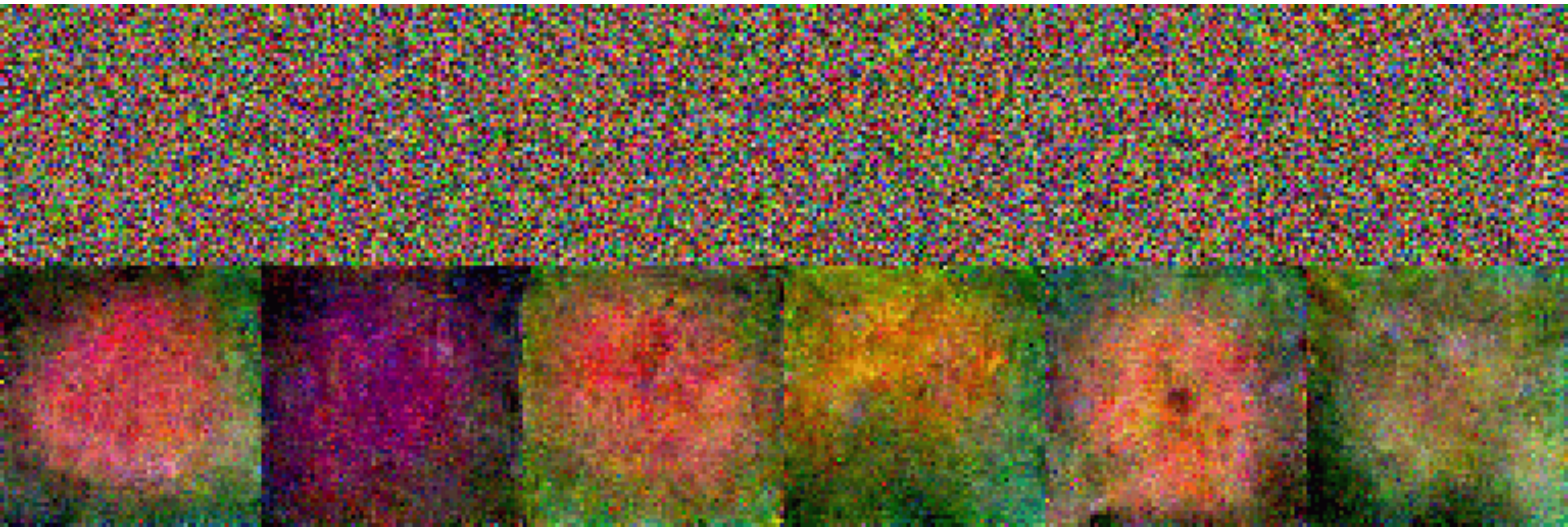
Source: <https://lilianweng.github.io/posts/2018-10-13-flow-models/>



GAN learning to generate images (linear time)



VAE learning to generate images (log time)



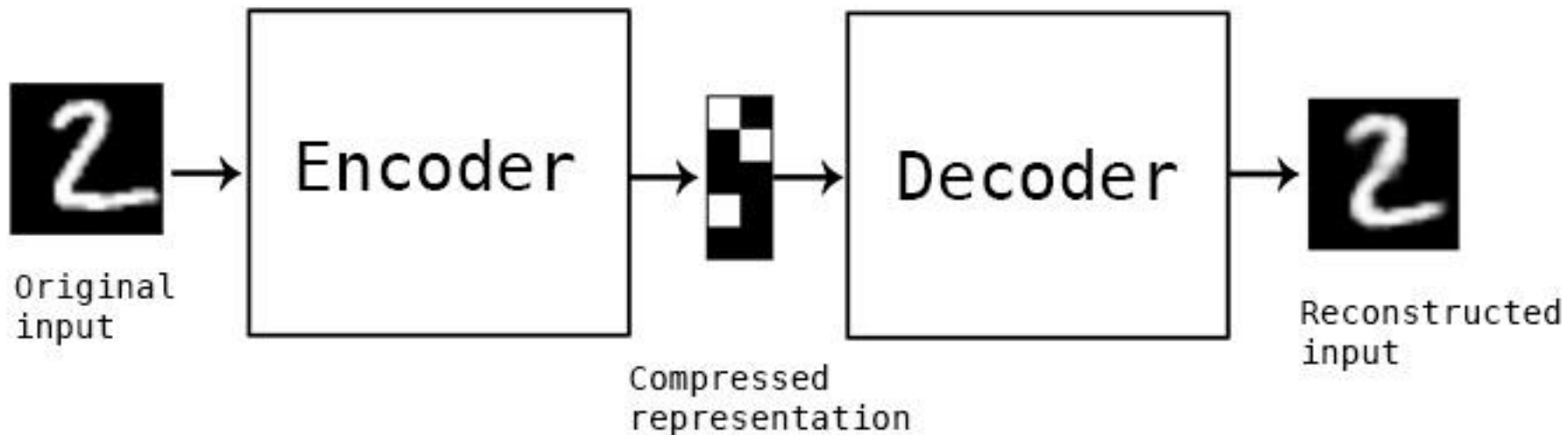
Denoising Diffusion Models learning to generate images

Variational Auto-Encoder

"Autoencoding" is a data compression algorithm.

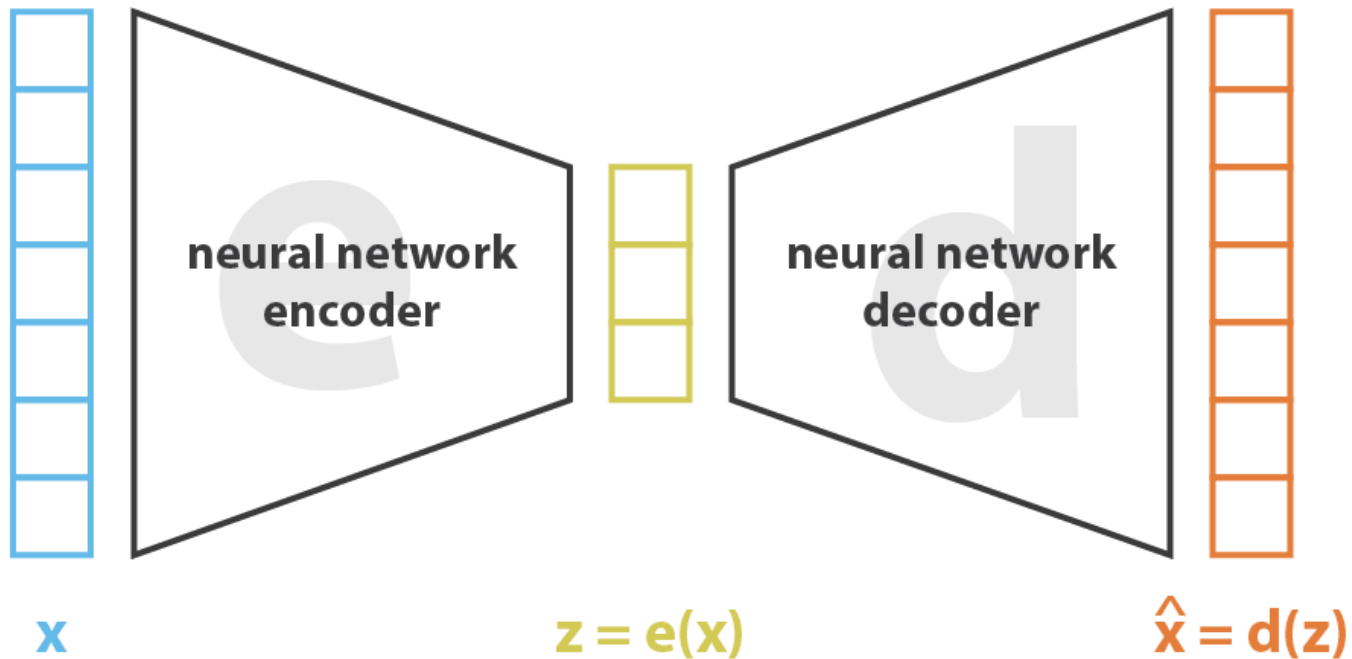
The compression and decompression functions are

1) data-specific, 2) lossy, and 3) *learned automatically from examples* rather than engineered by a human.



Source: <https://blog.keras.io/building-autoencoders-in-keras.html>

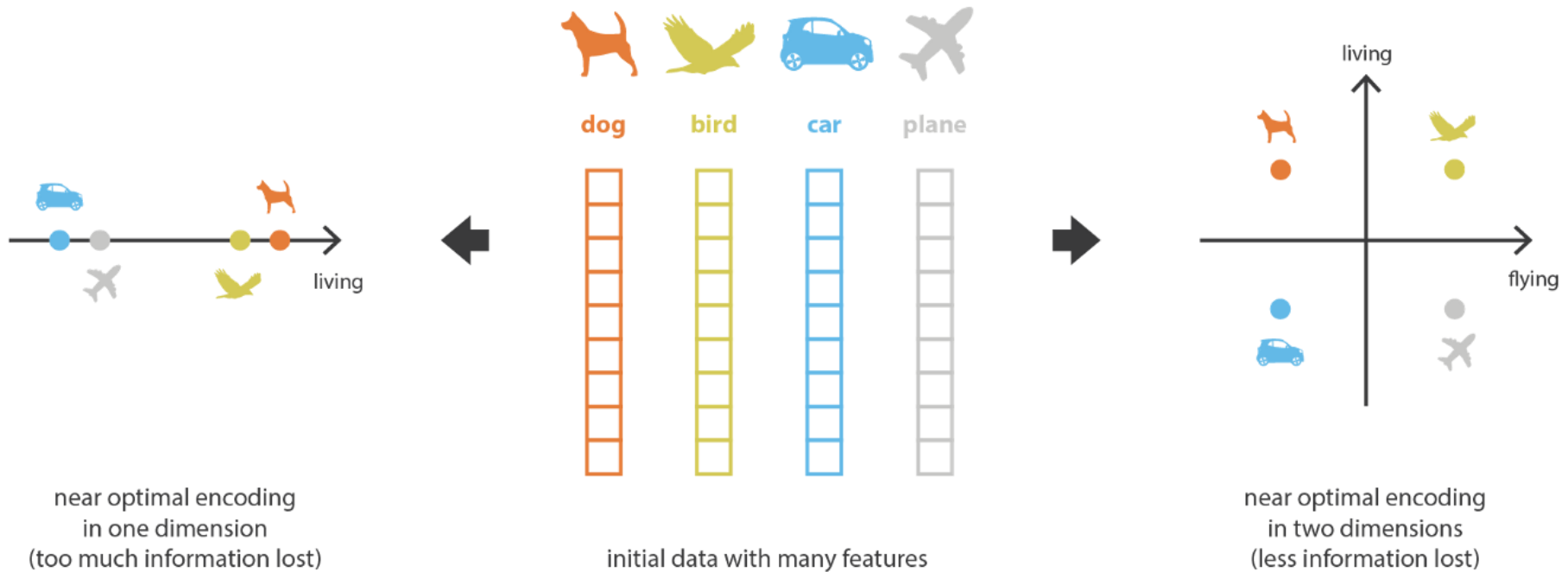
Source: <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>



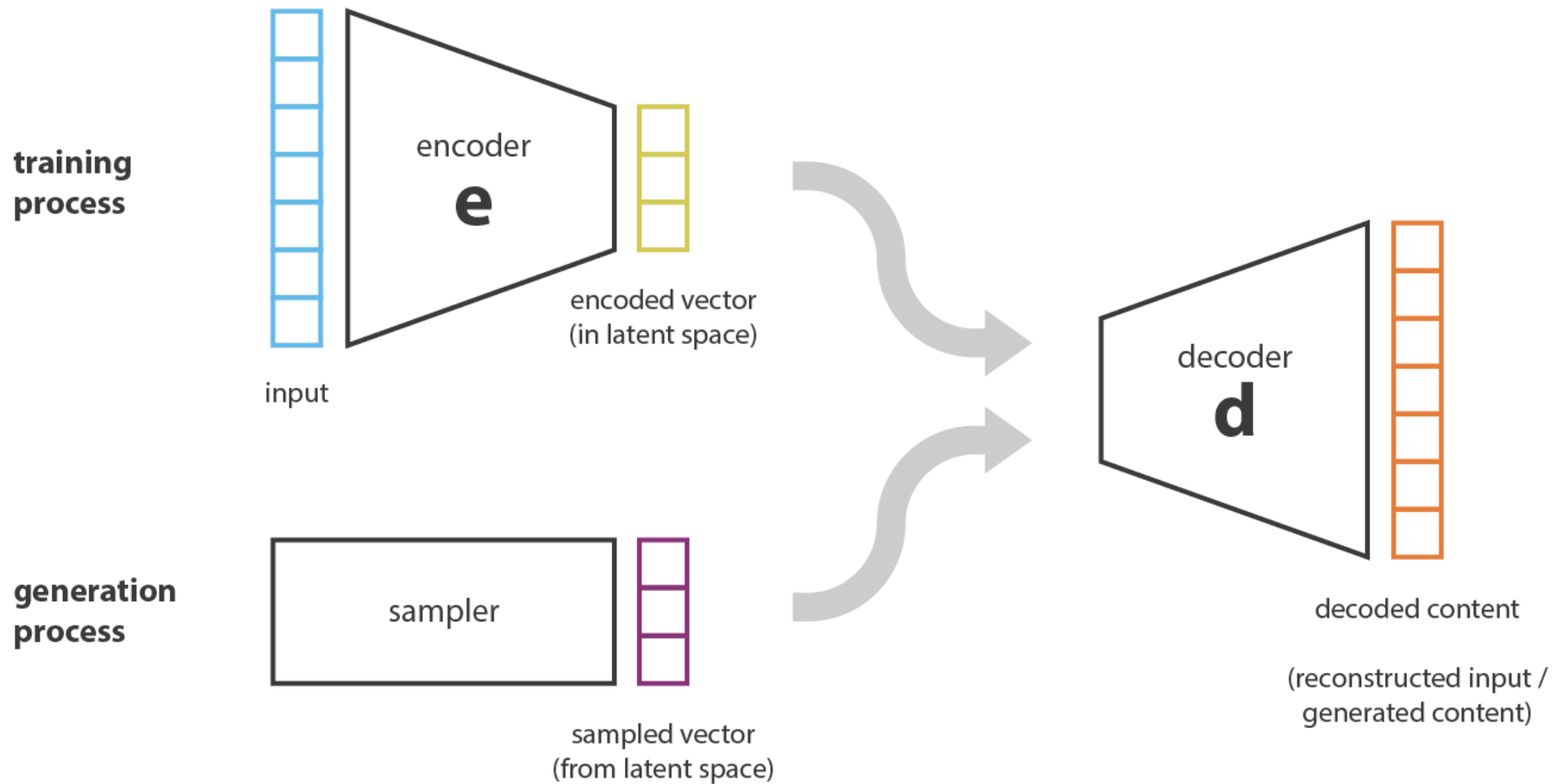
$$\text{loss} = ||x - \hat{x}||^2 = ||x - d(z)||^2 = ||x - d(e(x))||^2$$

Illustration of an autoencoder with its loss function.

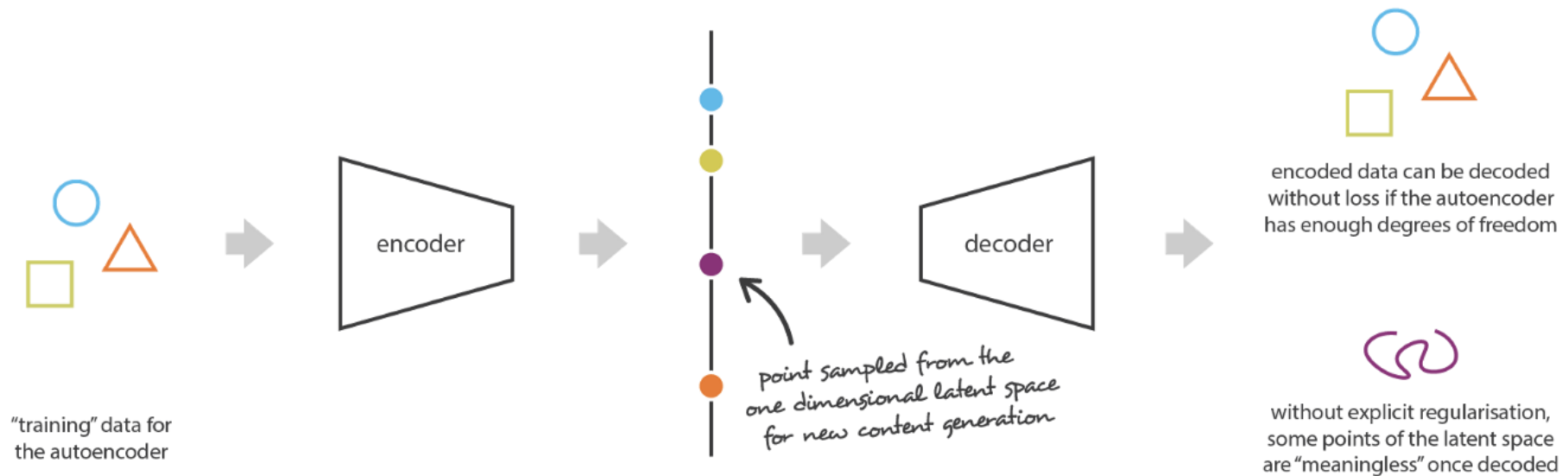
1. The more complex the architecture is, the more the autoencoder can proceed to a high dimensionality reduction while keeping reconstruction loss low.
2. An encoder with “infinite power” could theoretically takes our N initial data points and encodes them as 1, 2, 3, ... up to N (or more generally, as N integer on the real axis) and the associated decoder could make the reverse transformation, with no loss during the process.
3. The lack of interpretable and exploitable structures in the latent space (**lack of regularity**)



When reducing dimensionality, we want to keep the main structure there exists among the data.



Can we generate new data by decoding points that are randomly sampled from the latent space?



- Irregular latent space prevent us from using autoencoder for new content generation.
- The quality and relevance of generated data depend on the regularity of the latent space.



A “regular” and an “irregular” latent space.

Intuitions about the regularisation

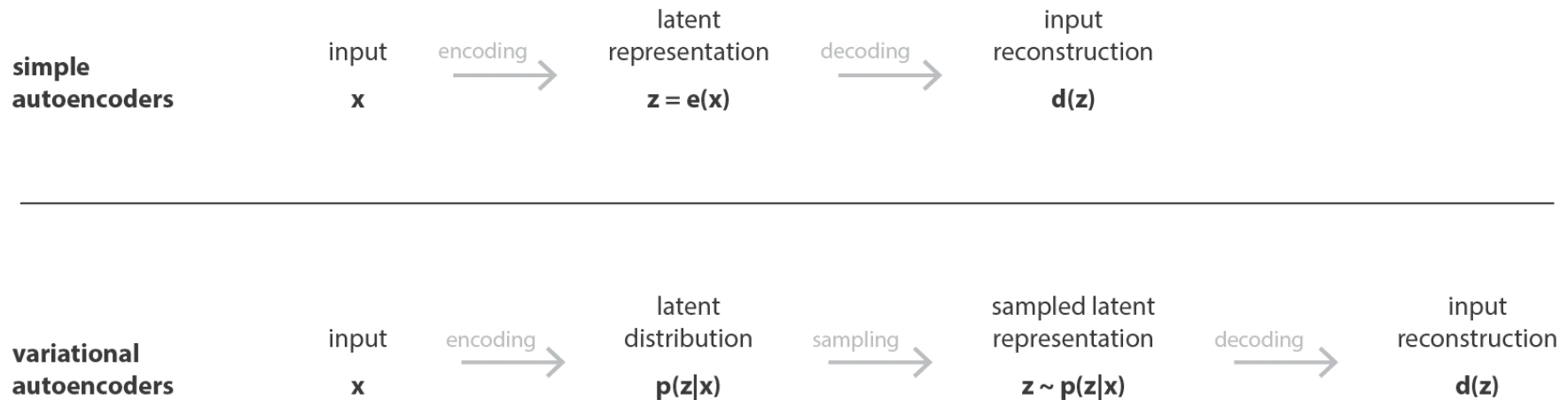
- **continuity** (two close points in the latent space should not give two completely different contents once decoded)
- **completeness** (for a chosen distribution, a point sampled from the latent space should give “meaningful” content once decoded)

- In order to be able to use the decoder of our autoencoder for generative purpose, one should make sure that **the latent space is regular enough**. One solution is to introduce **explicit regularisation** during the training process.
- A **variational autoencoder** can be defined as being **an autoencoder whose training is regularised** to avoid overfitting and ensure that the latent space has good properties that enable generative process.

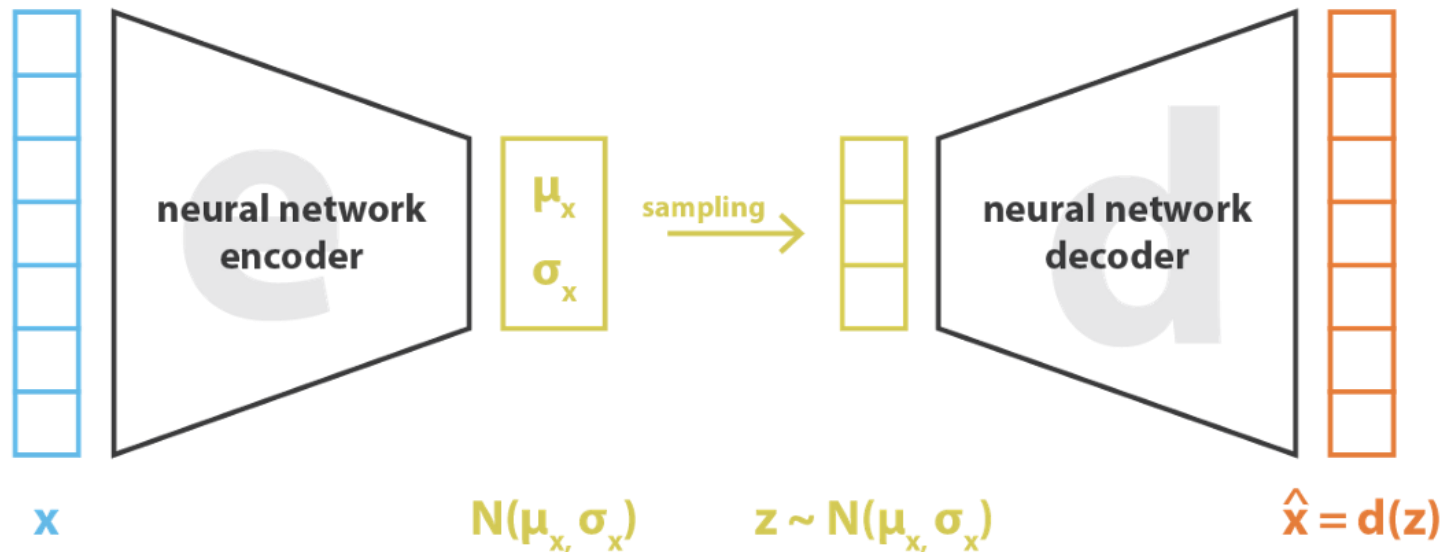
Reference:

Kingma, D. P., & Welling, M. **Auto-encoding variational bayes**. *ICLR (2014)*. **Citation: 49,000**

Instead of encoding an input as a **single point**, VAE encode it as **a distribution** over the latent space.



Latent distribution $p(z|x)$: **normal distribution**



$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

The loss function is composed of
a **reconstruction term** and a **regularisation term**.

To measure the distance between two distribution p and q , we use Kullback–Leibler divergence

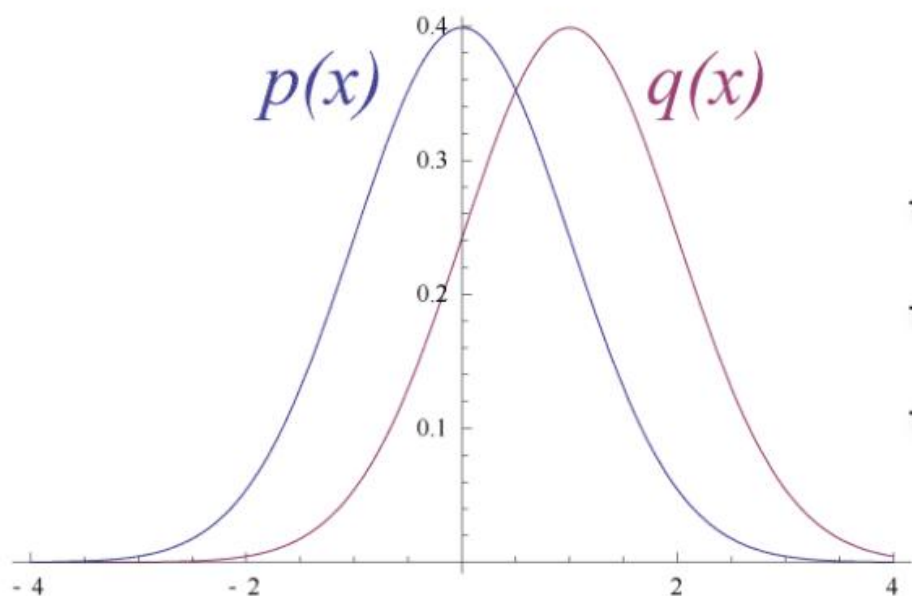
$$D_{KL}(p||q) = \int_x p(x) \log \frac{p(x)}{q(x)} dx$$

1. KL divergence is **non-negative**
2. KL divergence is **zero** if and only if **$p=q$**

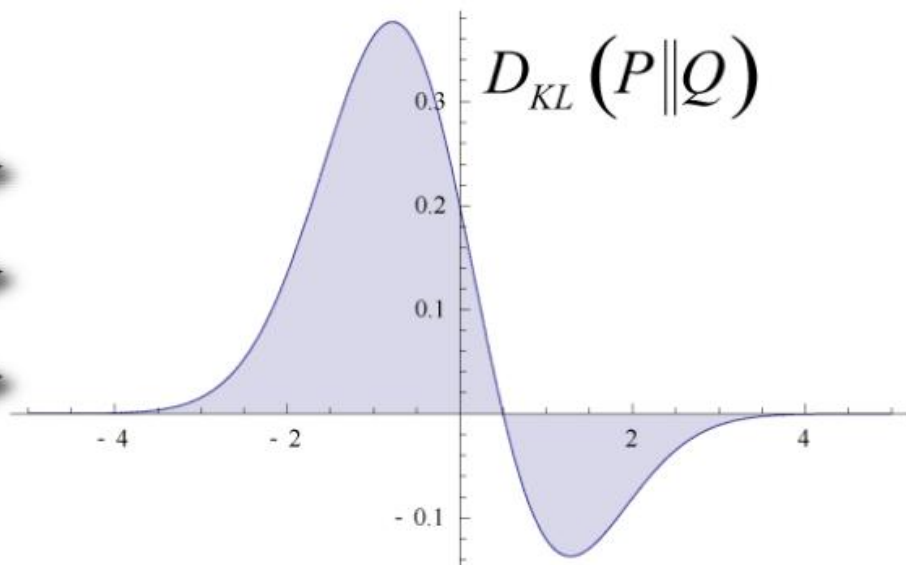
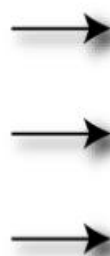
Multivariate normal distributions [\[edit\]](#)

Suppose that we have two [multivariate normal distributions](#), with means μ_0, μ_1 and with (non-singular) [covariance matrices](#) Σ_0, Σ_1 . If the two distributions have the same dimension, k , then the relative entropy between the distributions is as follows:[\[26\]](#)

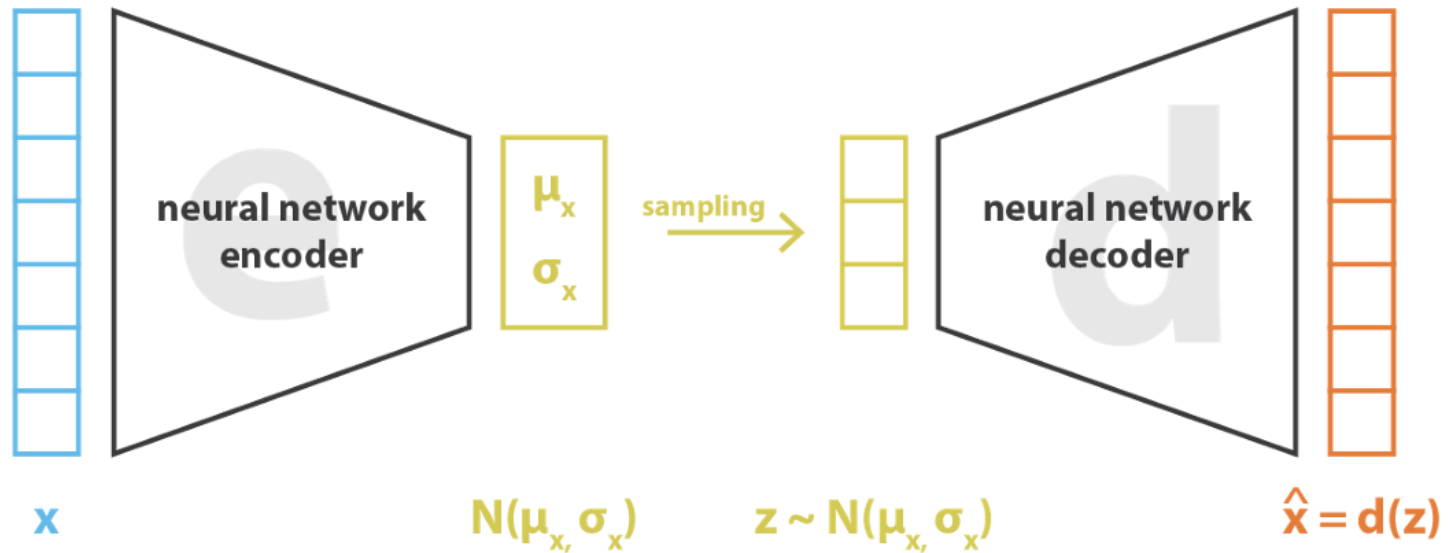
$$D_{\text{KL}}(\mathcal{N}_0 \parallel \mathcal{N}_1) = \frac{1}{2} \left(\text{tr}(\Sigma_1^{-1} \Sigma_0) - k + (\mu_1 - \mu_0)^{\top} \Sigma_1^{-1} (\mu_1 - \mu_0) + \ln \left(\frac{\det \Sigma_1}{\det \Sigma_0} \right) \right).$$



Original Gaussian PDF's

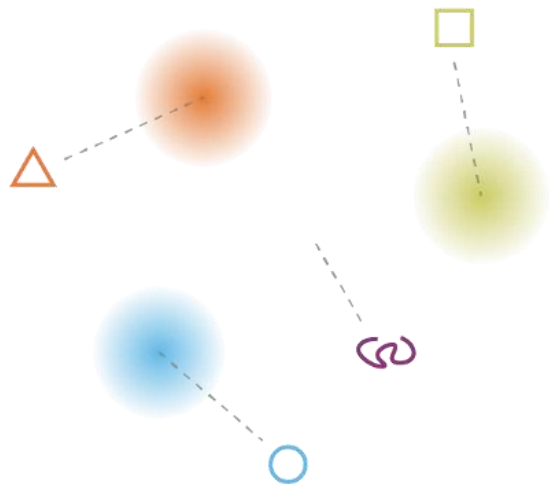


KL Area to be Integrated

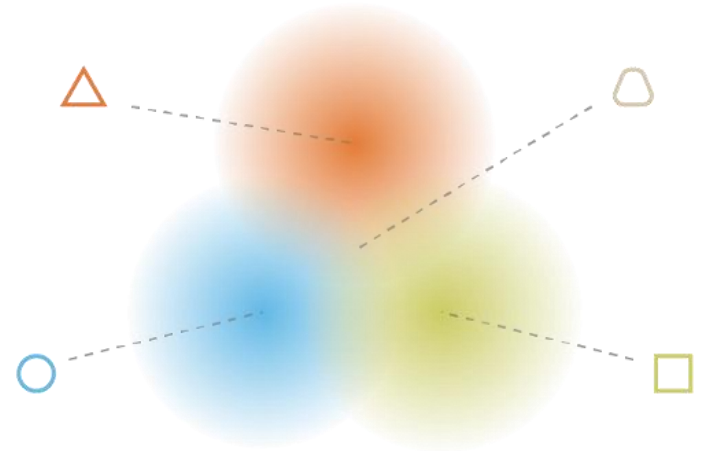


$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

The loss function is composed of
a **reconstruction term** and a **regularisation term**.

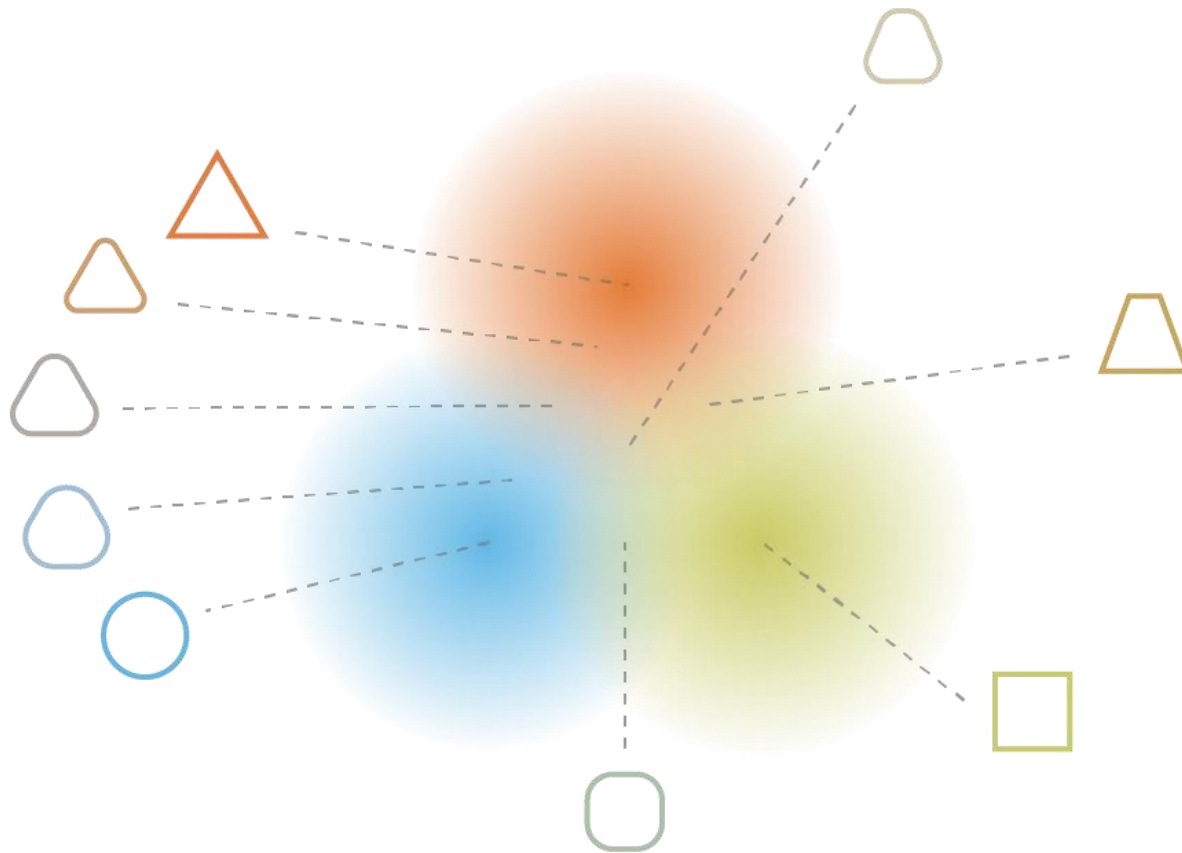


what can happen without regularisation



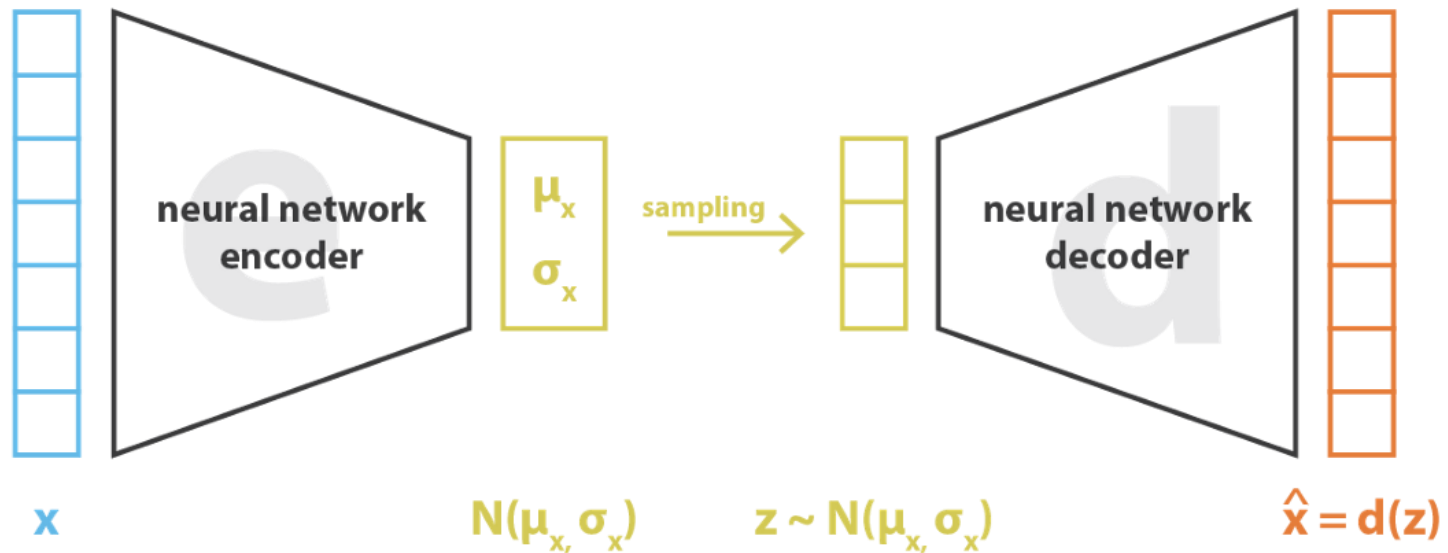
what we want to obtain with regularisation

The returned distributions of VAEs is regularised to obtain a latent space with good properties.



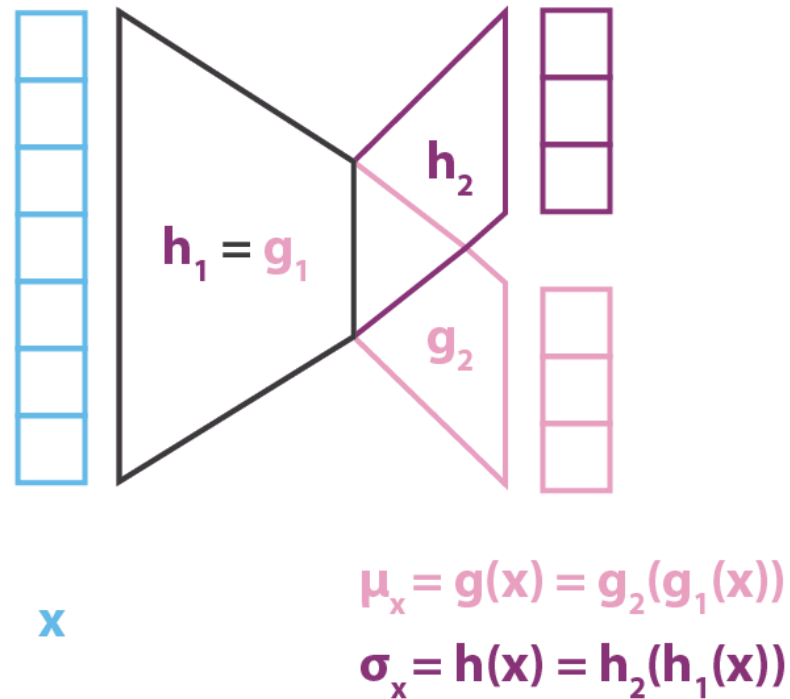
Regularisation tends to create a “gradient” over the information encoded in the latent space.

Neural Network Implementation



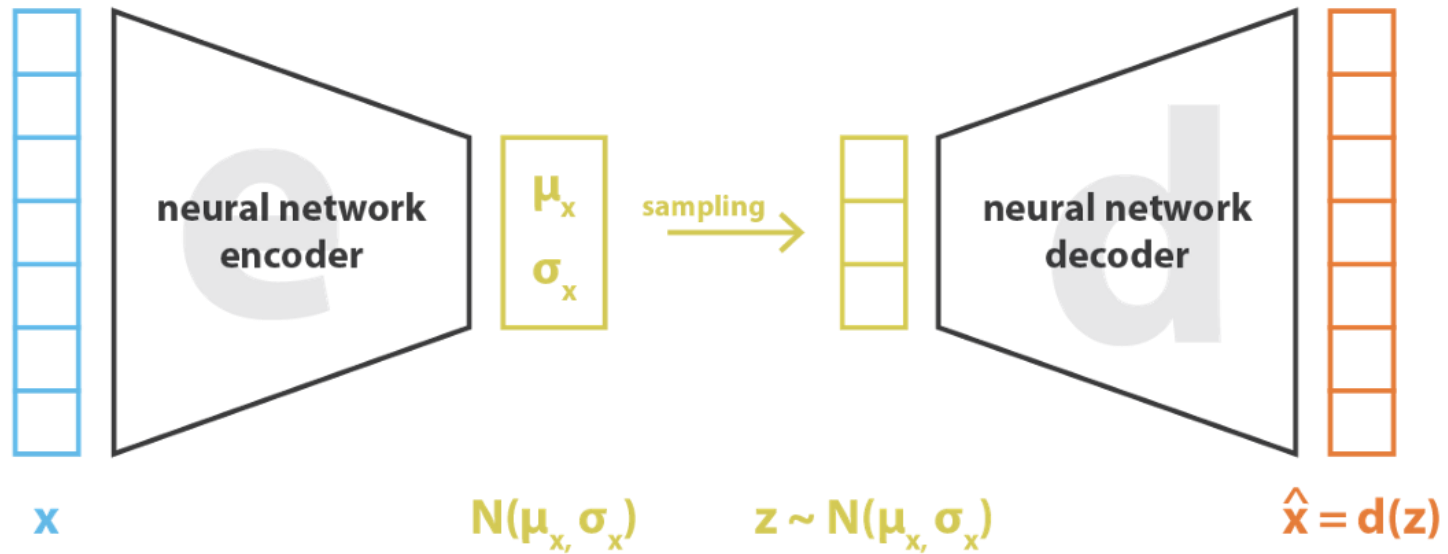
$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

Neural Network Implementation



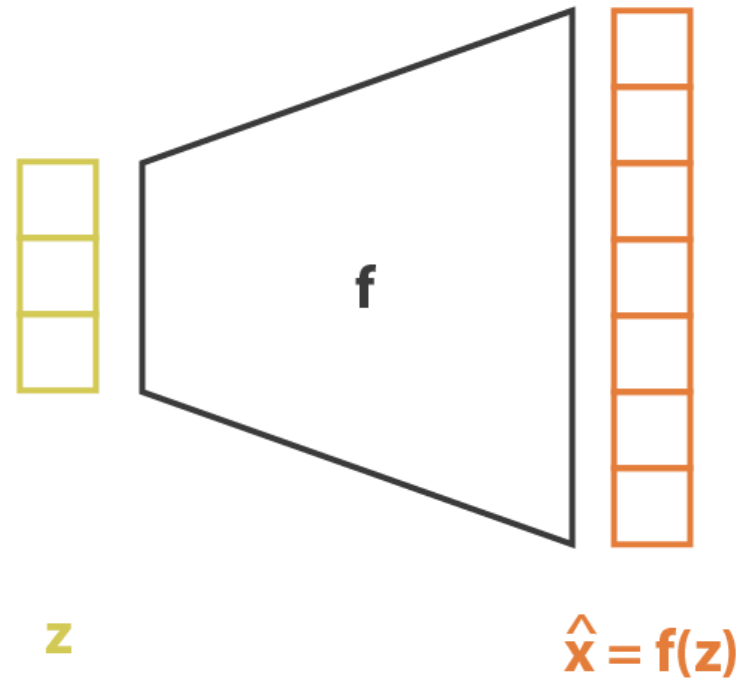
Encoder neural network outputs **Mean** and **Covairance**

Neural Network Implementation



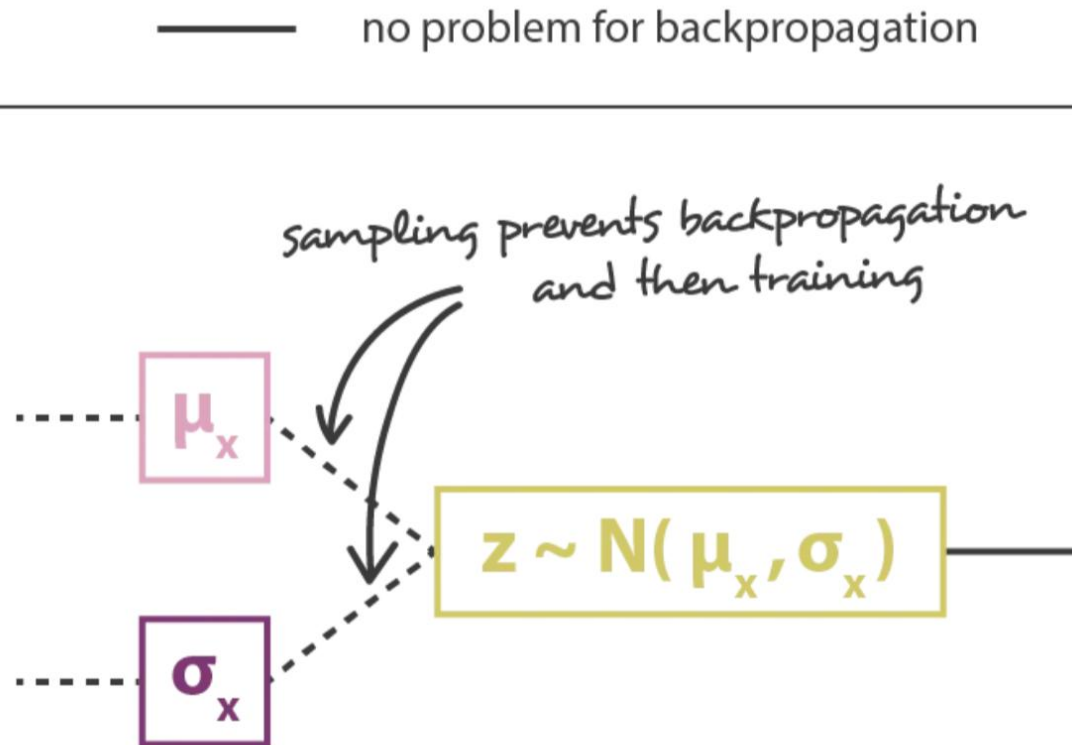
Sampling latent variable z based on
Mean and Covariance from encoder

Neural Network Implementation



Decoder neural network outputs **reconstruction**

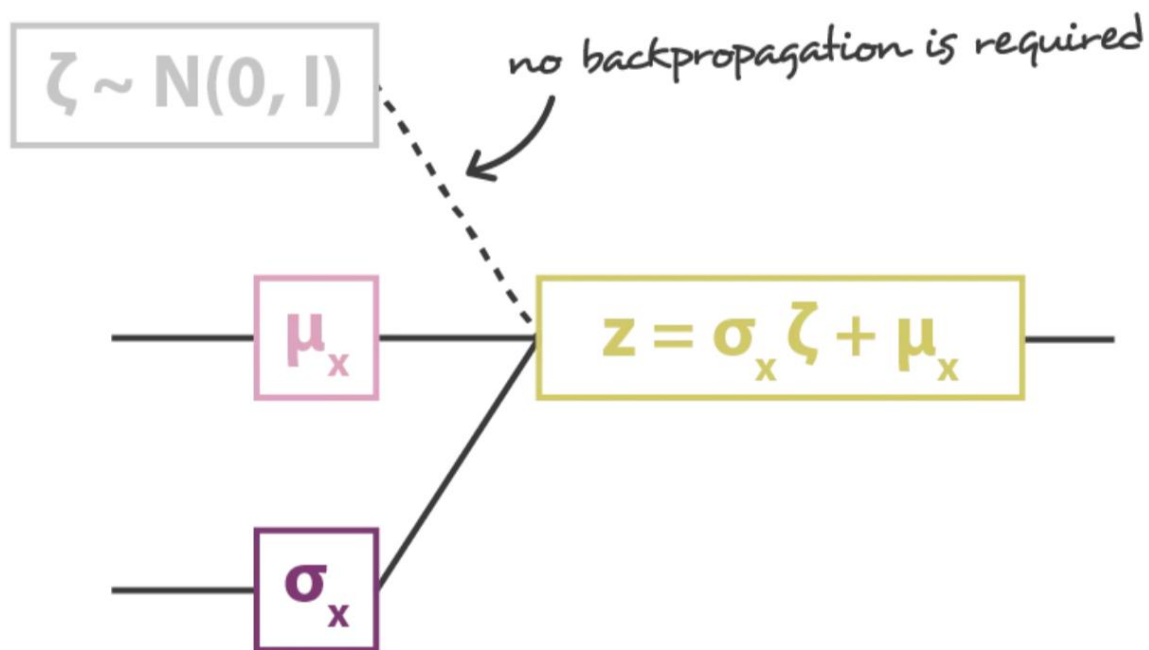
Sampling prevents backpropagation and the training

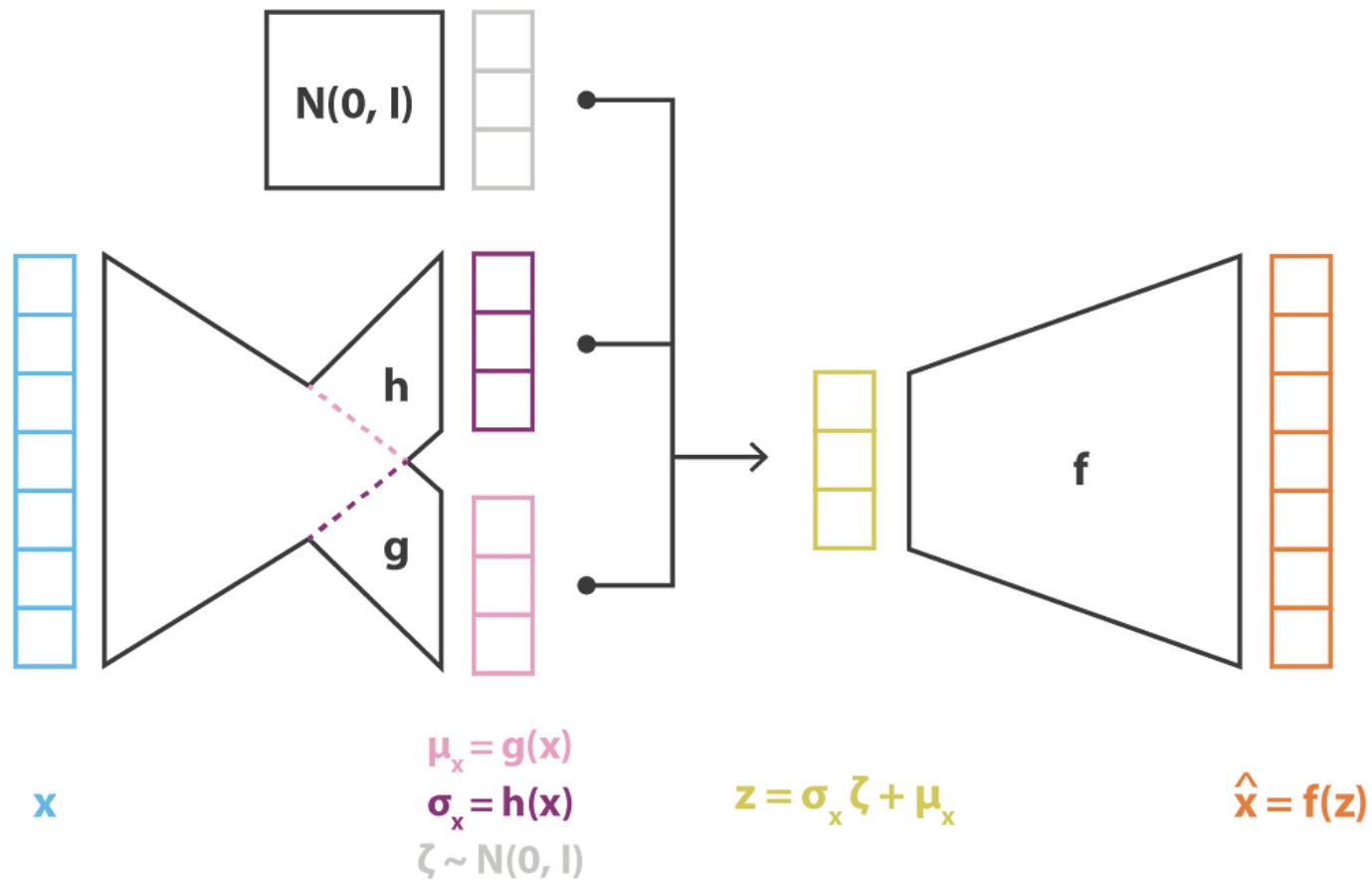


Reparameterization trick

$$z = \sigma_x \zeta + \mu_x \quad \zeta \sim N(0, I)$$

..... backpropagation is not possible due to sampling

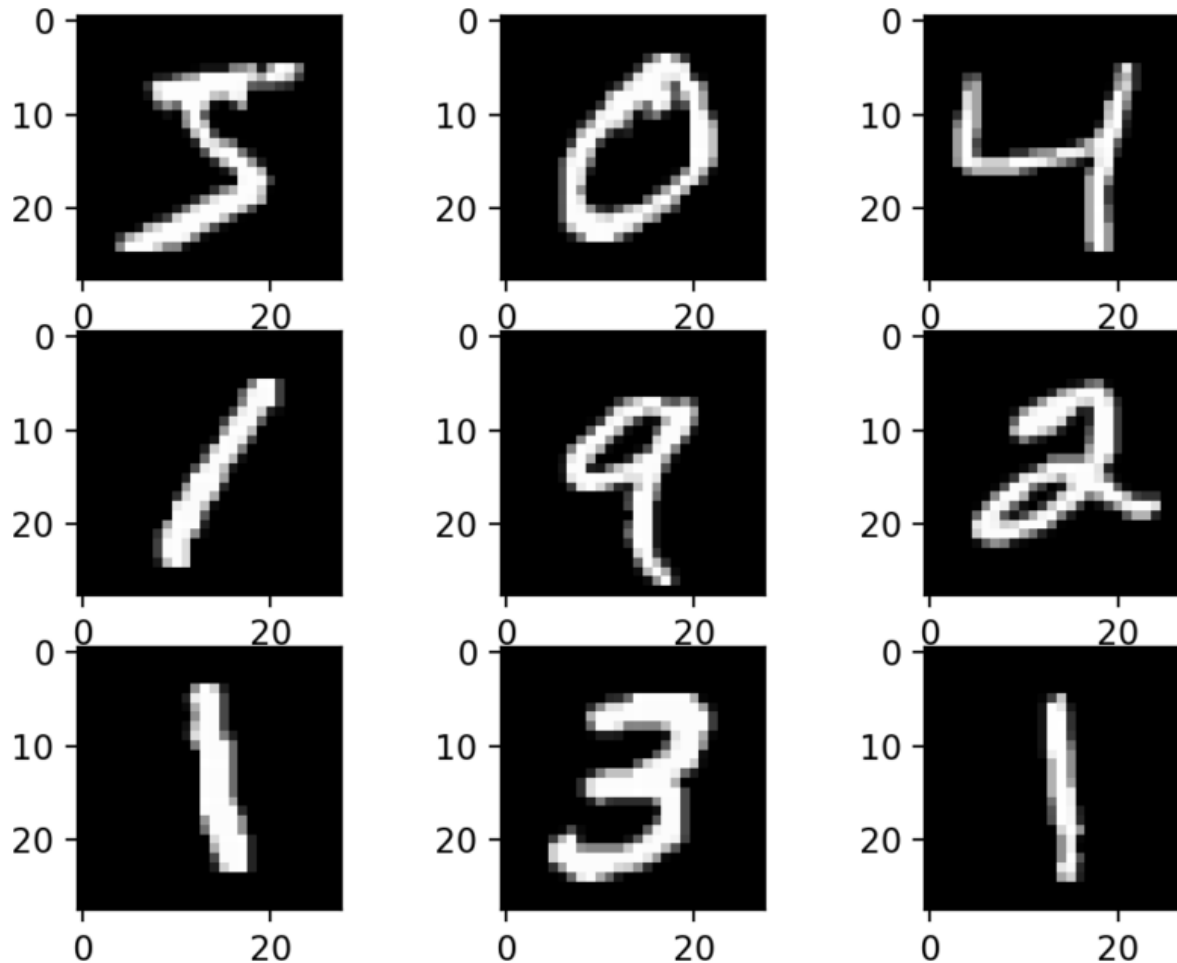




$$\text{loss} = C ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = C ||x - f(z)||^2 + \text{KL}[N(g(x), h(x)), N(0, I)]$$

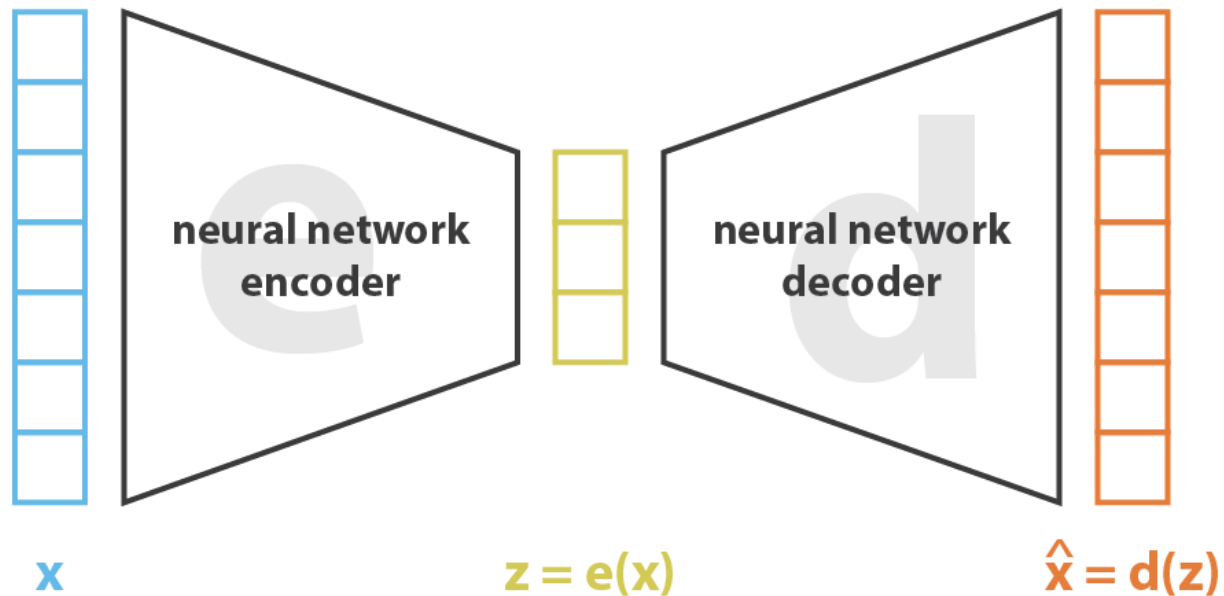
- Variational autoencoders (VAEs) are autoencoders that tackle the problem of the latent space irregularity.
- VAE makes the encoder return a distribution over the latent space instead of a single point.
- VAE loss function includes a regularisation term over the returned distribution in order to ensure a better organisation of the latent space.

Data: 28*28 black-white images with label



Source: <https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/#:~:text=MNIST%20Handwritten%20Digit%20Classification%20Dataset,-The%20MNIST%20dataset&text=It%20is%20a%20dataset%20of,from%200%20to%209%2C%20inclusively.>

#1 Train an Autoencoder on MNIST



$$\text{loss} = ||x - \hat{x}||^2 = ||x - d(z)||^2 = ||x - d(e(x))||^2$$

Define the encoder

```
class Encoder(nn.Module):
    def __init__(self, latent_dims):
        super(Encoder, self).__init__()
        self.linear1 = nn.Linear(784, 512)
        self.linear2 = nn.Linear(512, latent_dims)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.linear1(x))
        return self.linear2(x)
```

Define the decoder

```
class Decoder(nn.Module):
    def __init__(self, latent_dims):
        super(Decoder, self).__init__()
        self.linear1 = nn.Linear(latent_dims, 512)
        self.linear2 = nn.Linear(512, 784)

    def forward(self, z):
        z = F.relu(self.linear1(z))
        z = torch.sigmoid(self.linear2(z))
        return z.reshape((-1, 1, 28, 28))
```

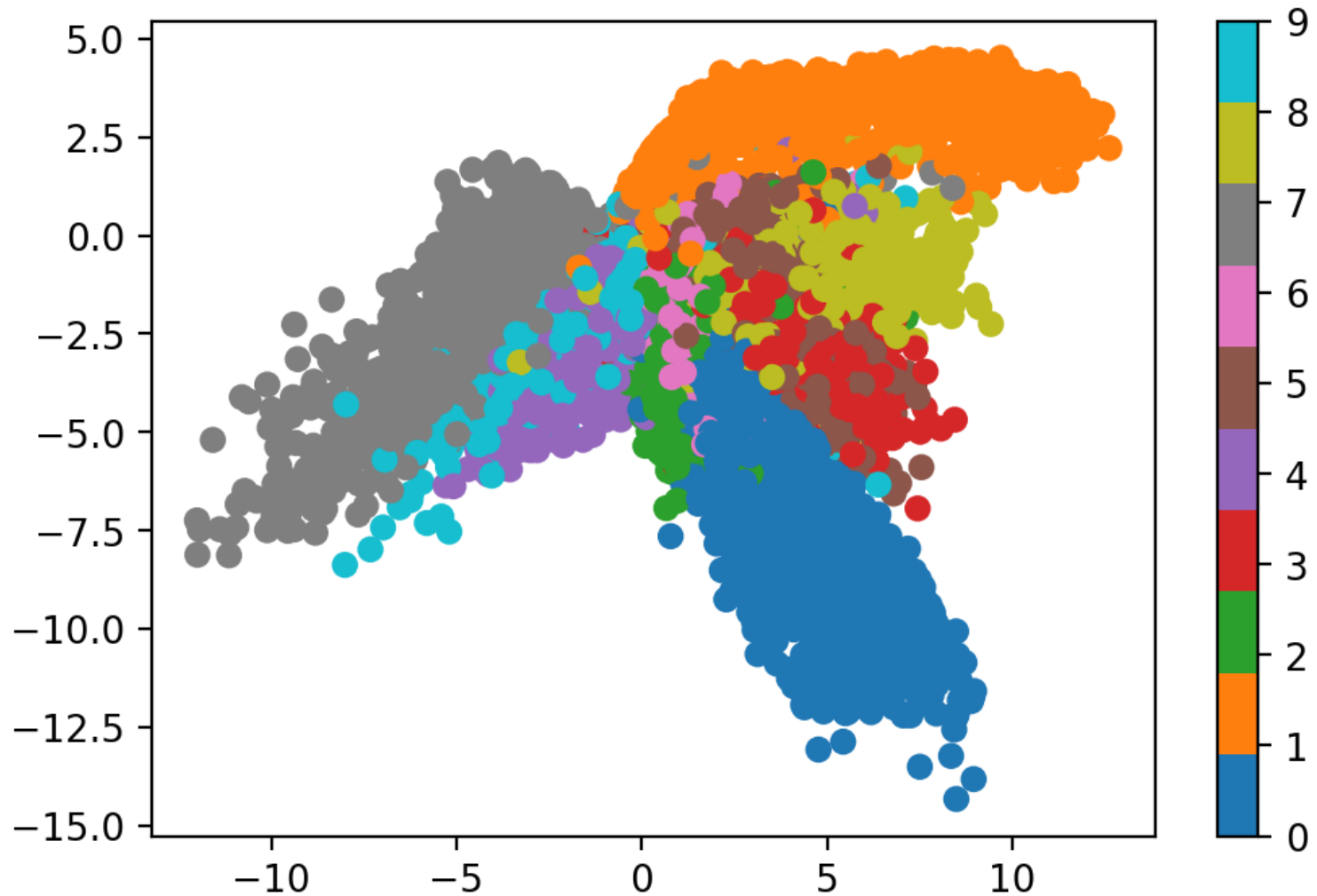
Define the auto-encoder

```
class Autoencoder(nn.Module):  
    def __init__(self, latent_dims):  
        super(Autoencoder, self).__init__()  
        self.encoder = Encoder(latent_dims)  
        self.decoder = Decoder(latent_dims)  
  
    def forward(self, x):  
        z = self.encoder(x)  
        return self.decoder(z)
```

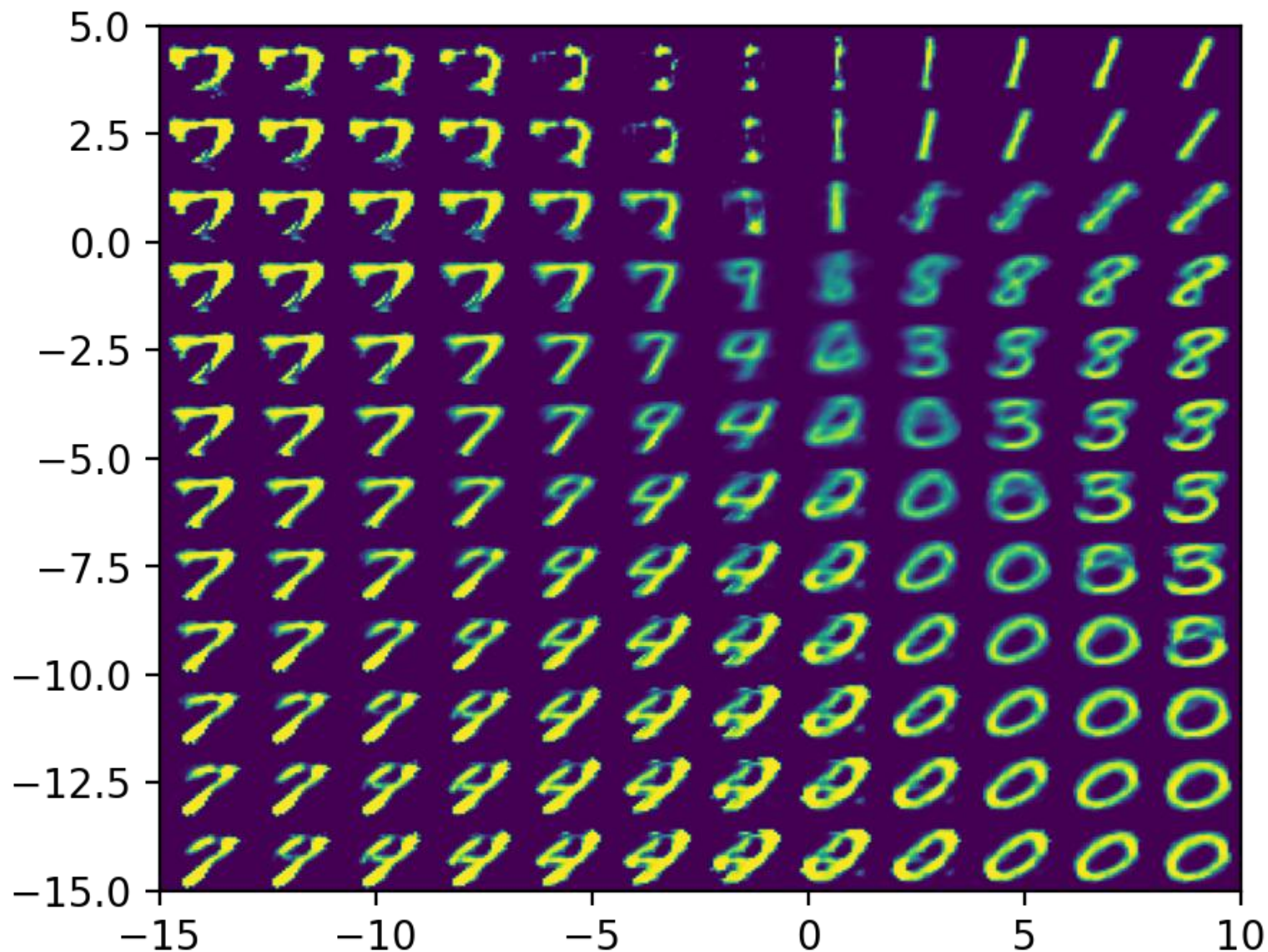
Train the auto-encoder

```
def train(autoencoder, data, epochs=20):
    opt = torch.optim.Adam(autoencoder.parameters())
    for epoch in range(epochs):
        for x, y in data:
            opt.zero_grad()
            x_hat = autoencoder(x)
            loss = ((x - x_hat)**2).sum()
            loss.backward()
            opt.step()
        print('Epoch: %d'%epoch, "\ is finised.")
    return autoencoder
```

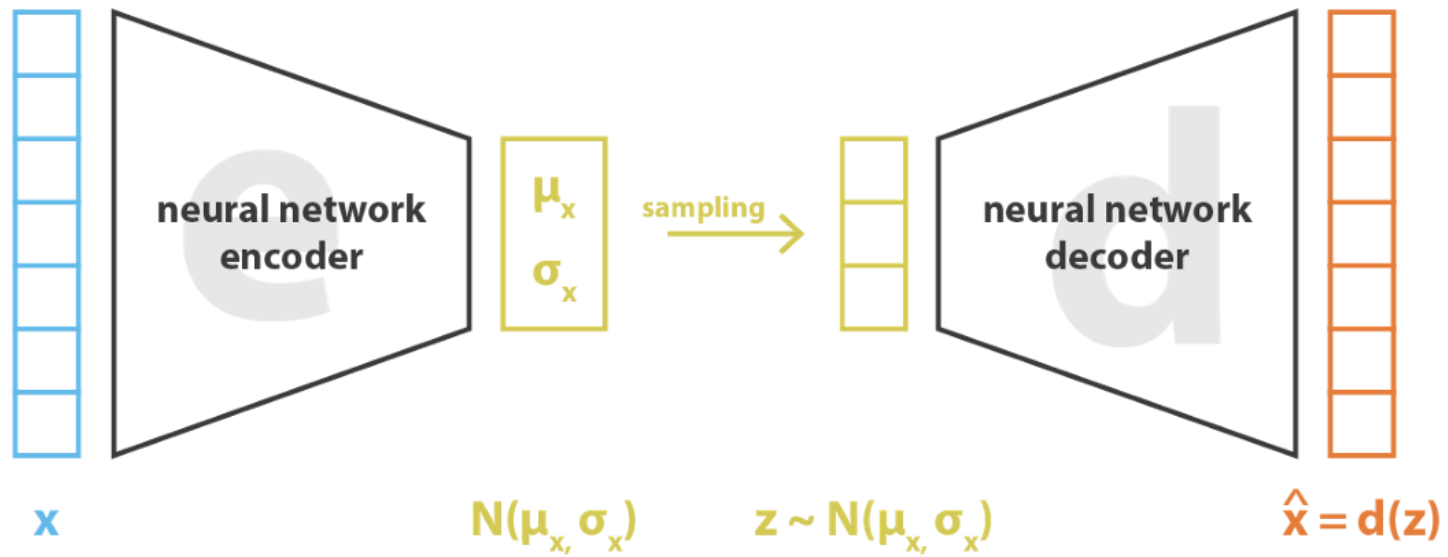
Visualization of 2D Latent space



Visualization of decoded 2D Latent space



#2 Train an VAE on MNIST



$$\text{loss} = ||x - \hat{x}||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = ||x - d(z)||^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

Define the Variational encoder

```
class VariationalEncoder(nn.Module):
    def __init__(self, latent_dims):
        super(VariationalEncoder, self).__init__()
        self.linear1 = nn.Linear(784, 512)
        self.linear2 = nn.Linear(512, latent_dims)
        self.linear3 = nn.Linear(512, latent_dims)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.linear1(x))
        mu = self.linear2(x)
        sigma = torch.exp(self.linear3(x))
        return mu, sigma
```

Define the Variational auto-encoder

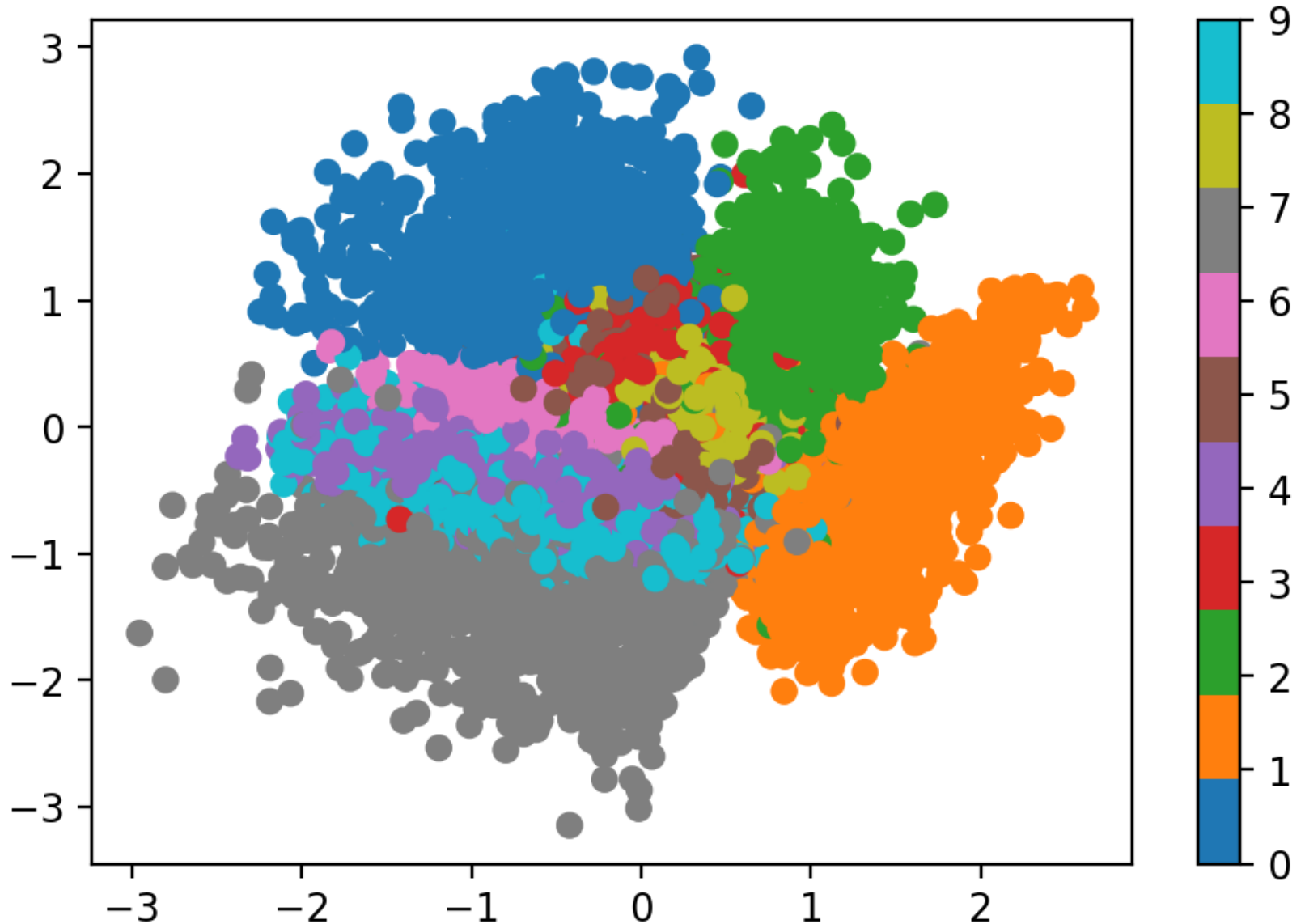
```
class VariationalAutoencoder(nn.Module):
    def __init__(self, latent_dims):
        super(VariationalAutoencoder, self).__init__()
        self.encoder = VariationalEncoder(latent_dims)
        self.decoder = Decoder(latent_dims)
        self.N = torch.distributions.Normal(0, 1)

    def forward(self, x):
        mu, sigma = self.encoder(x)
        z = mu + sigma*self.N.sample(mu.shape)
        self.kl = (sigma**2 + mu**2 - torch.log(sigma) - 1/2).sum()
        return self.decoder(z)
```

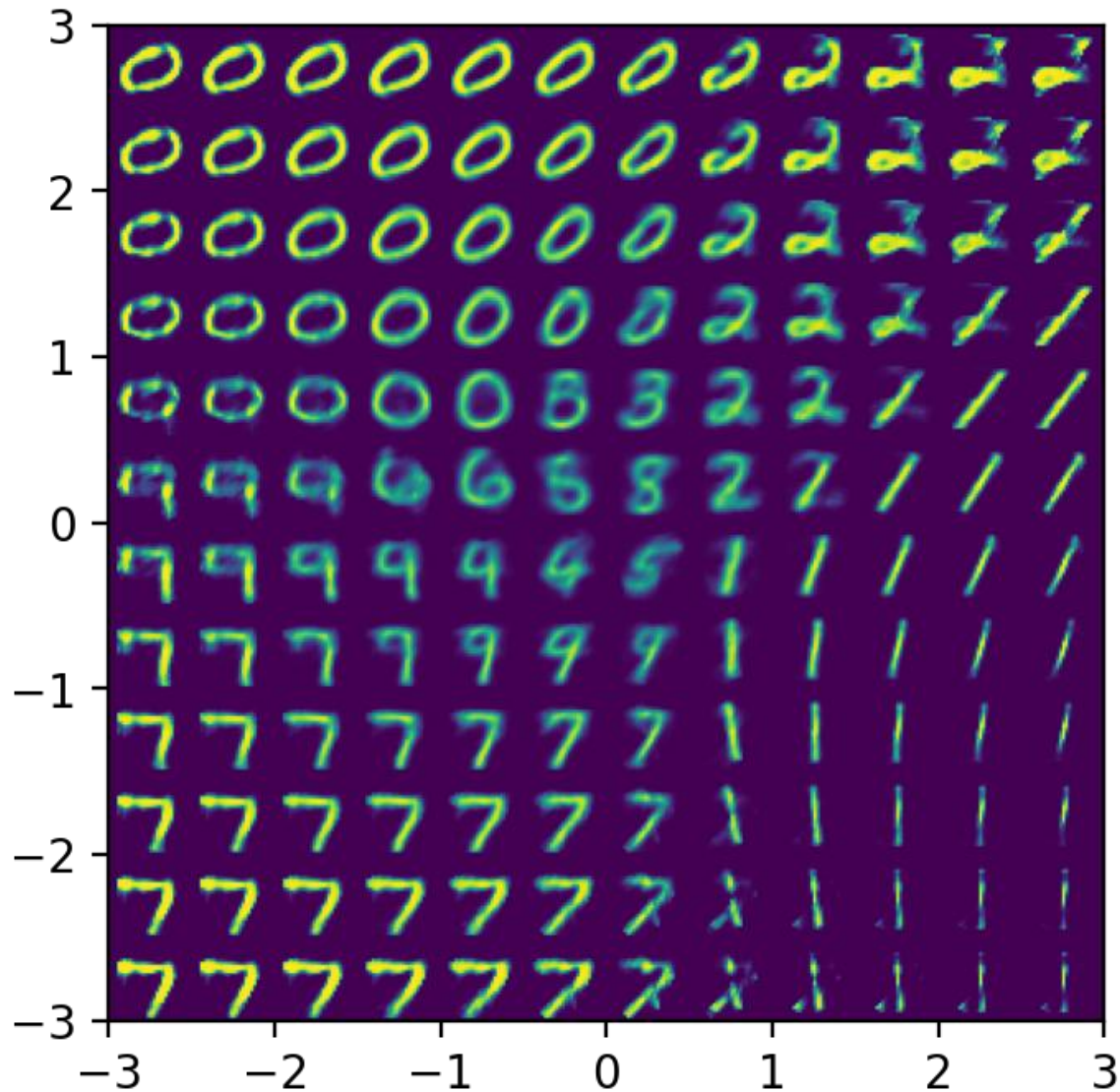
Train the VAE

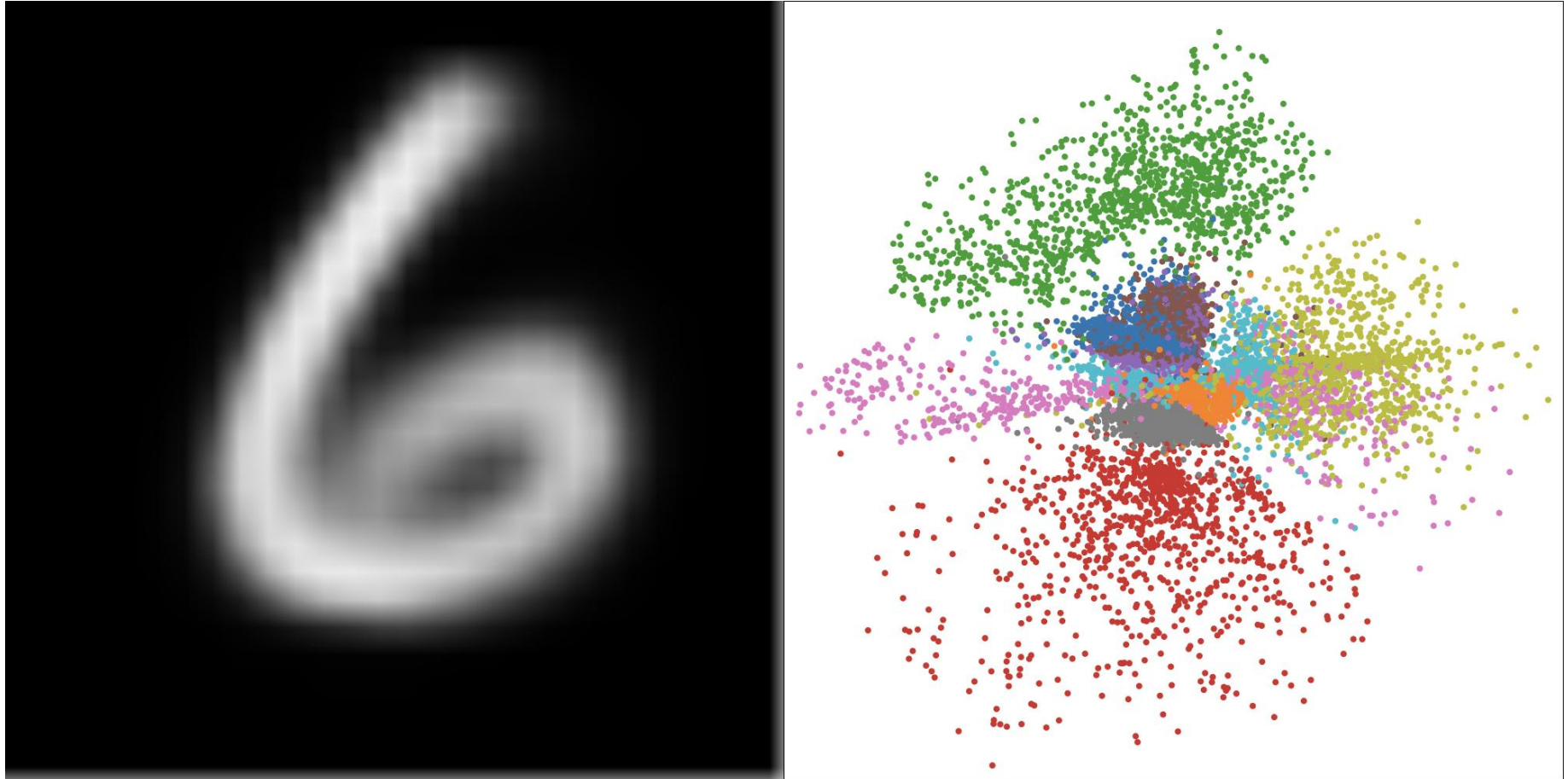
```
def train(autoencoder, data, epochs=20):
    opt = torch.optim.Adam(autoencoder.parameters())
    for epoch in range(epochs):
        for x, y in data:
            #x = x.to(device)
            opt.zero_grad()
            x_hat = autoencoder(x)
            loss = ((x - x_hat)**2).sum() + autoencoder.kl
            loss.backward()
            opt.step()
        print("Epoch %d"%epoch, "\ is finished.")
    return autoencoder
```

Visualization of 2D Latent space



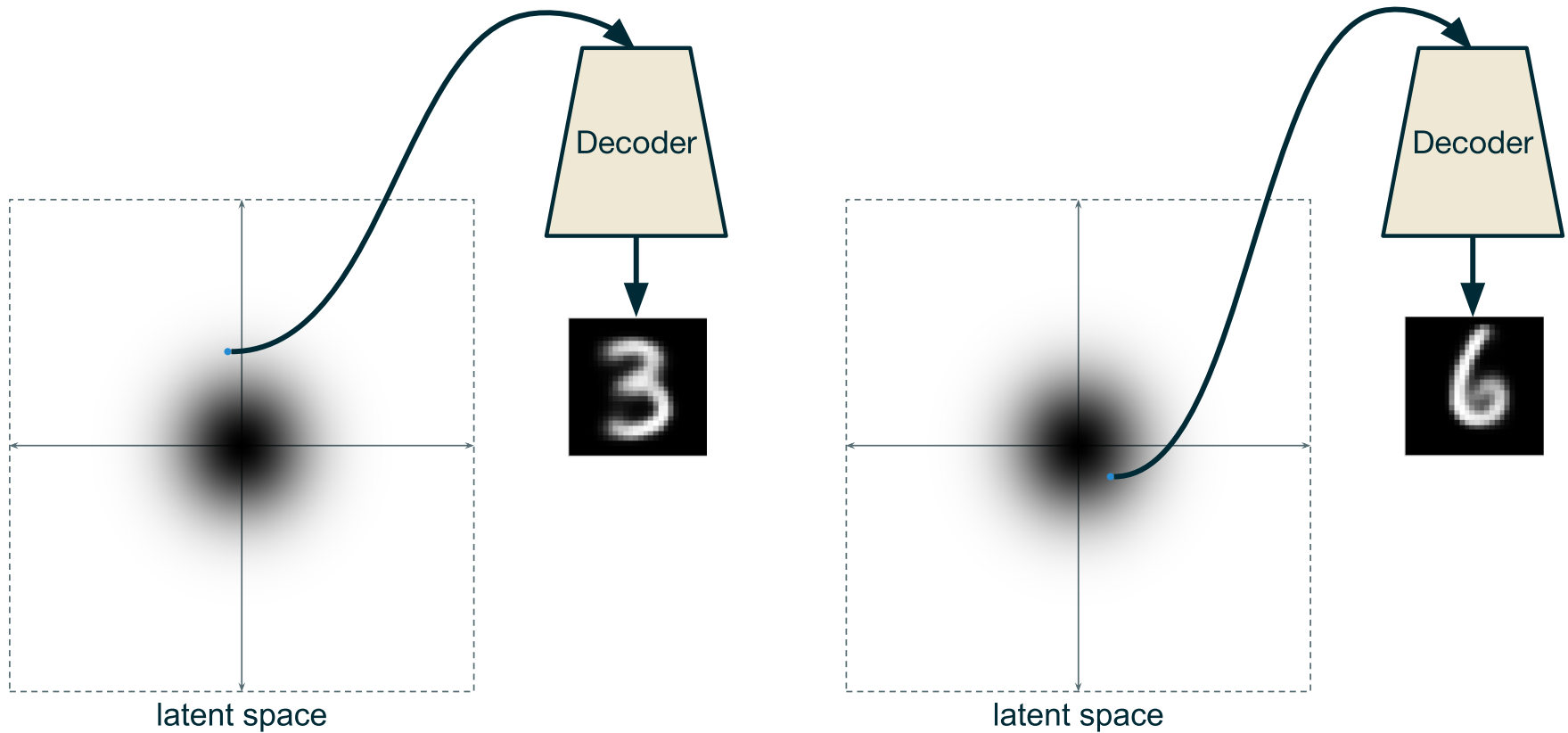
Visualization of decoded 2D Latent space





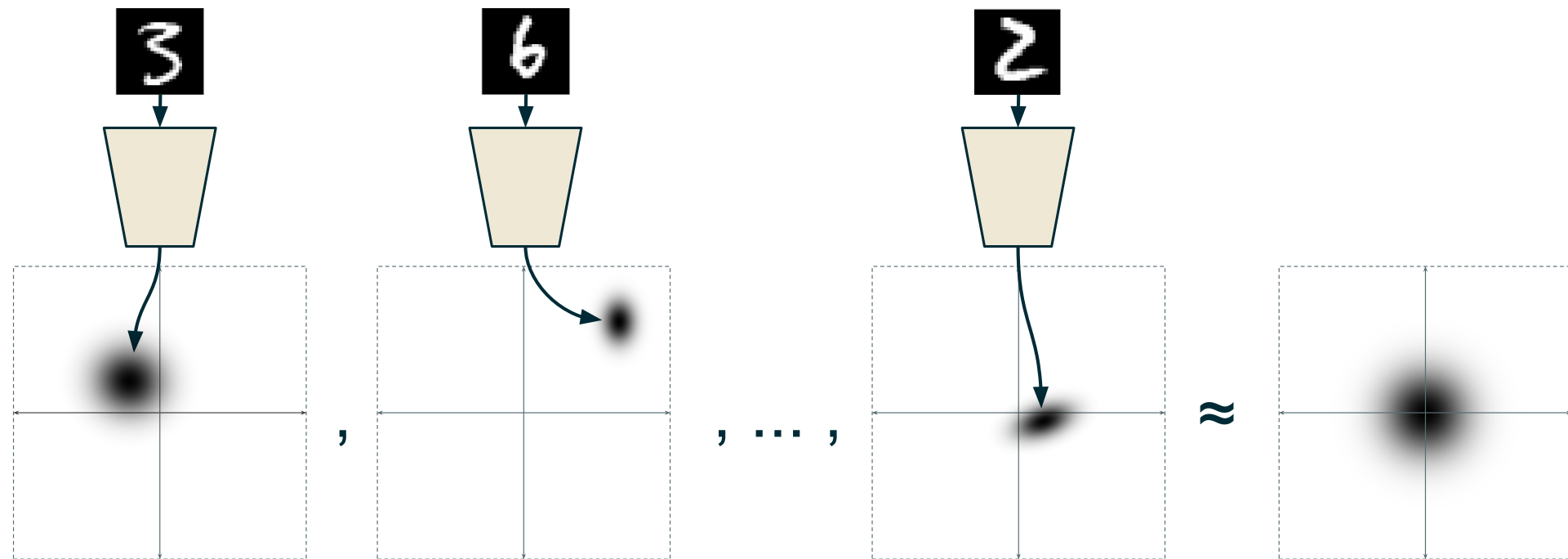
VAE Latent Space Explorer

The decoder of VAE cannot produce an image of a particular number on demand.



Source: <https://ijdykeman.github.io/ml/2016/12/21/cvae.html>

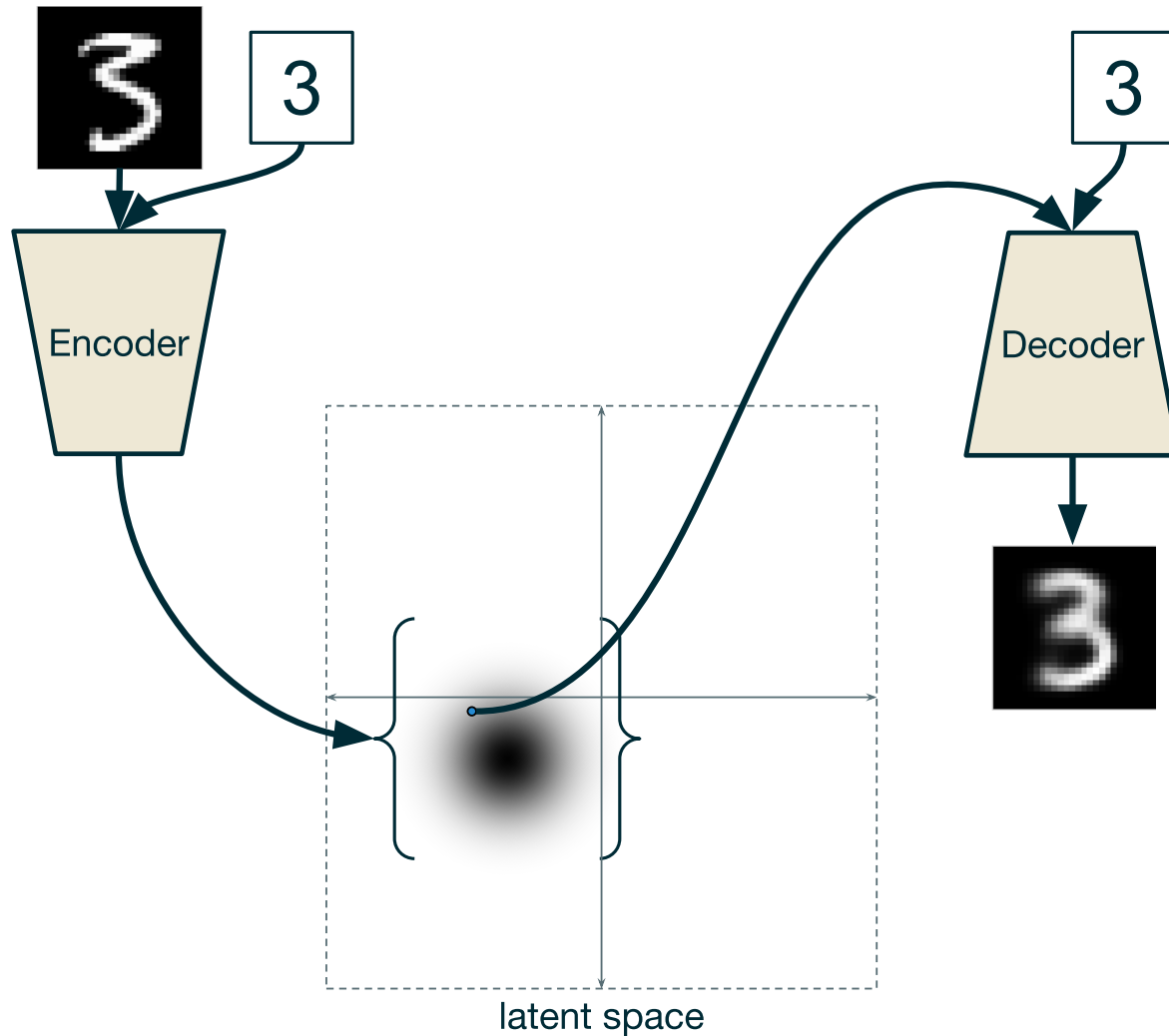
The decoder of VAE cannot produce an image of a particular number on demand.



Source: <https://ijdykeman.github.io/ml/2016/12/21/cvae.html>

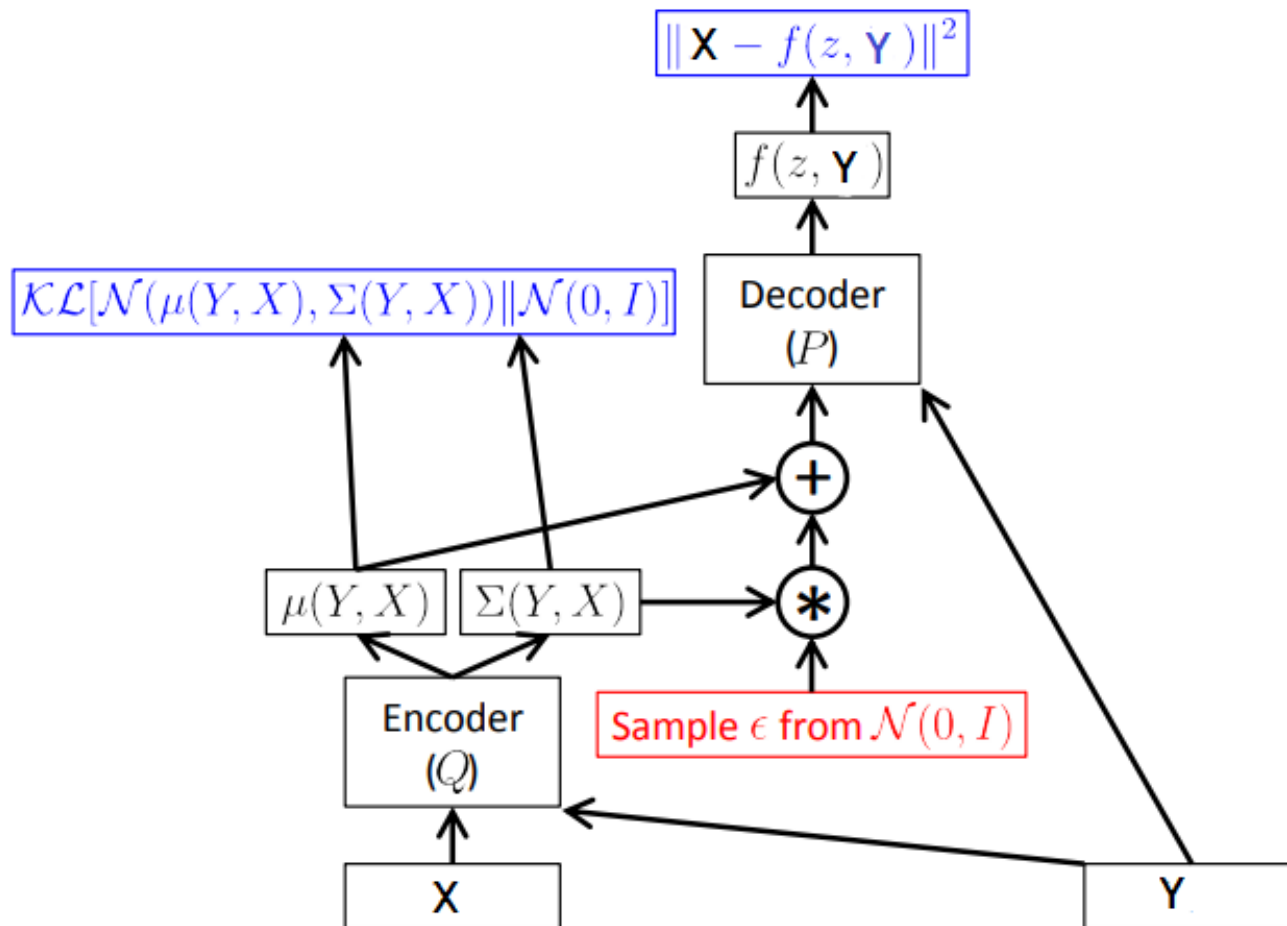
Conditional variational autoencoder (CVAE)

has an extra input to both the encoder and the decoder.

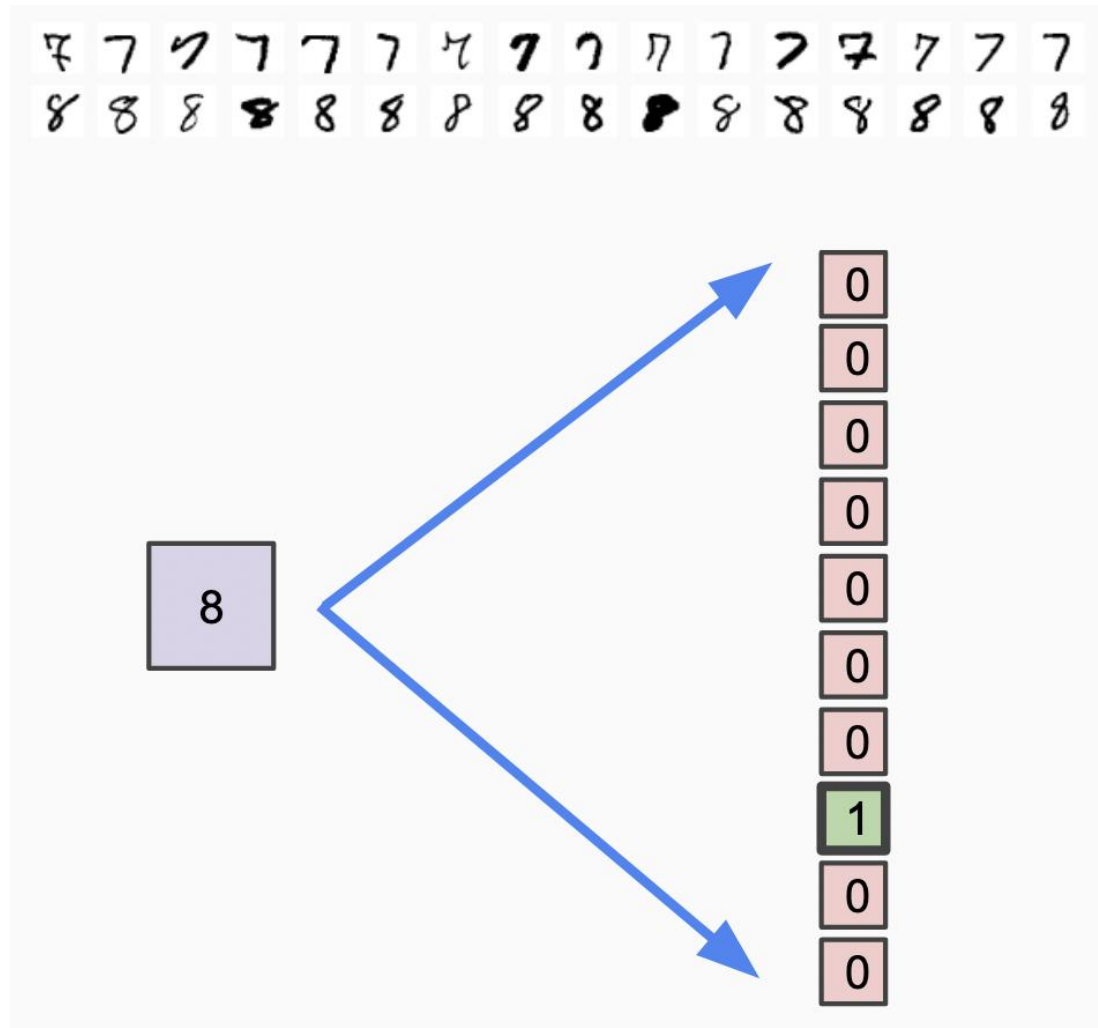


Conditional variational autoencoder (CVAE)

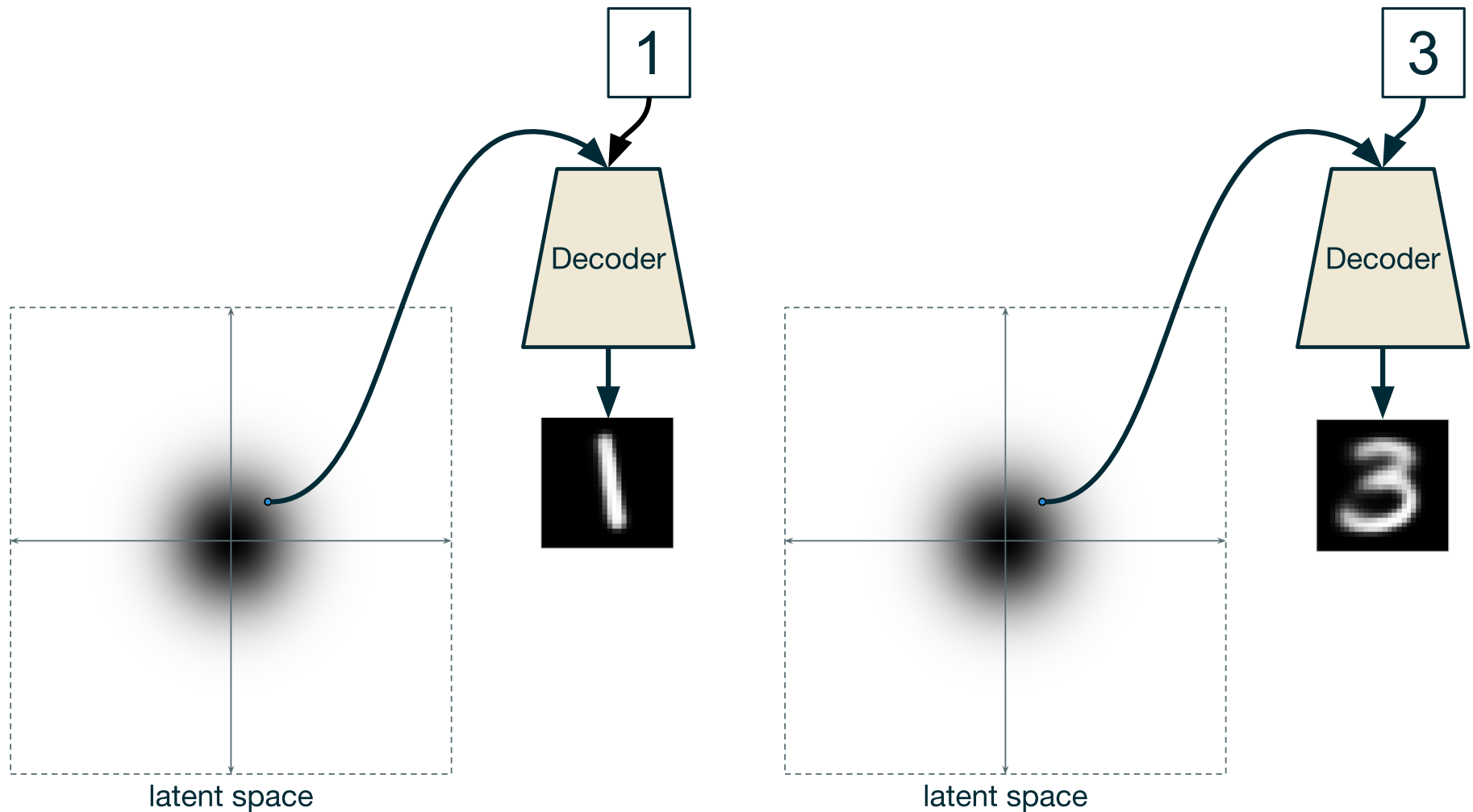
has an extra input to both the encoder and the decoder.



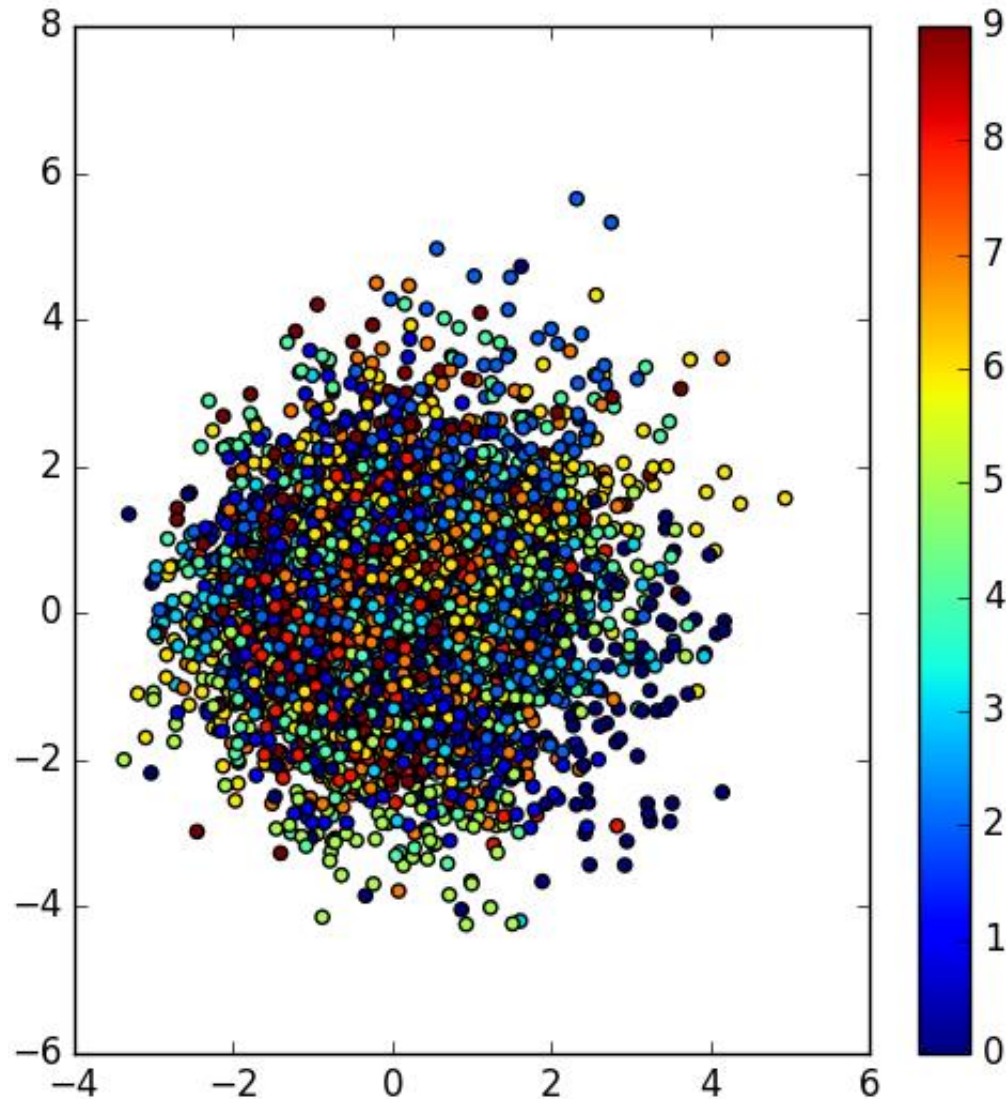
At training time, the label of the image is provided to the encoder and decoder. Label is represented as a one-hot vector.



A Conditional VAE generating images according to given labels.



Visualization of 2D Latent space of Conditional VAE



Source: <https://agustinus.kristia.de/techblog/2016/12/17/conditional-vae/>

Generated images of Conditional VAE by different classes

